

Heart Disease Risk Prediction

End-to-End MLOps Pipeline

Machine Learning Operations (MLOps) — AIMLCZG523
Assignment 01

Name: Chaitanya Kumar Goddu

BITS ID: 2024ac05880@wilp.bits-pilani.ac.in

GitHub Repository:

<https://github.com/ChaitanyaKumarGoddu/BITSMLOPSAssignment>

Video walkthrough:

https://github.com/ChaitanyaKumarGoddu/BITSMLOPSAssignment/raw/main/ShortVideo/pipeline_walkthrough.mp4

1. Project Overview

This project builds a machine-learning classifier that predicts the risk of heart disease from patient clinical data and deploys it as a cloud-ready, monitored REST API, following modern MLOps practice end to end: data acquisition and EDA, feature engineering, multi-model training with hyper-parameter tuning and cross-validation, MLflow experiment tracking, reproducible packaging, automated testing with GitHub Actions CI/CD, Docker containerisation, Kubernetes deployment, and Prometheus/Grafana monitoring.

Dataset: UCI Heart Disease (Cleveland), 303 patient records, 13 clinical features, binary target (presence/absence of disease).

Headline result: the tuned Logistic Regression pipeline achieves ROC-AUC 0.968, accuracy 0.90, and recall 0.93 on the held-out test set — recall being the priority metric for a screening tool where a missed case is costly.

Technology stack

Python 3.14 · pandas / scikit-learn / XGBoost · MLflow (SQLite backend) · FastAPI + Uvicorn · Pytest · Docker (multi-stage) · Kubernetes (Docker Desktop) + Helm · Prometheus + Grafana · GitHub Actions.

2. Setup & Installation

Requires Python 3.11-3.14. All commands are run from the repository root.

Source files:

- requirements.txt (runtime deps)
- requirements-dev.txt (+ EDA, tests, lint, notebook)

Commands:

```
python -m venv .venv
.venv\Scripts\Activate.ps1      # Windows PowerShell
pip install --upgrade pip
pip install -r requirements-dev.txt
```

Dependencies are pinned to wheel-available versions verified on Python 3.14 (Windows) and Python 3.12 (Linux CI). The runtime Docker image uses only requirements.txt; requirements-dev.txt adds EDA, testing, linting and Jupyter.

3. Task 1 - Data Acquisition & Exploratory Data Analysis

Source files:

- `src/config.py` - paths + feature schema (single source of truth)
- `src/data/download.py` - fetch dataset (ucimlrepo id=45, URL fallback)
- `src/data/preprocess.py` - `clean()`: coerce numeric, binarise target, de-dup
- `src/data/eda.py` - generate the EDA figures
- `notebooks/01_eda.ipynb` - narrated EDA notebook

`download.py` fetches the Cleveland dataset via the official `ucimlrepo` package (id=45), falling back to a direct download of `processed.cleveland.data`. `preprocess.clean()` coerces every column to numeric, binarises the 0-4 severity field into a 0/1 target (0 = no disease, ≥ 1 = disease), and drops exact duplicates. Missing values (the UCI '?' marker) appear only in `ca` (4 rows) and `thal` (2 rows) and are imputed later inside the modelling pipeline rather than dropped. The two classes are close to balanced (~46% positive).

Commands:

```
python -m src.data.download # -> data/raw/heart_disease_raw.csv (303 rows)
python -m src.data.preprocess # -> data/processed/heart_disease_clean.csv
python -m src.data.eda # -> reports/figures/eda_*.png
```

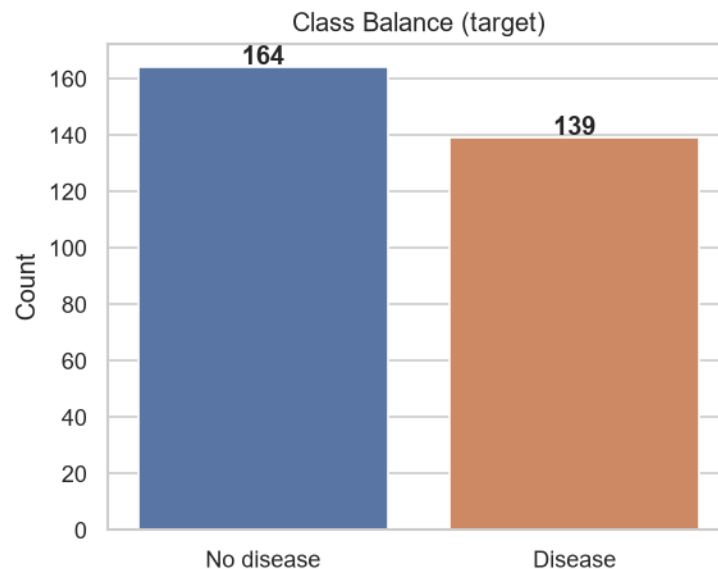


Figure 1. Class balance (~46% positive).

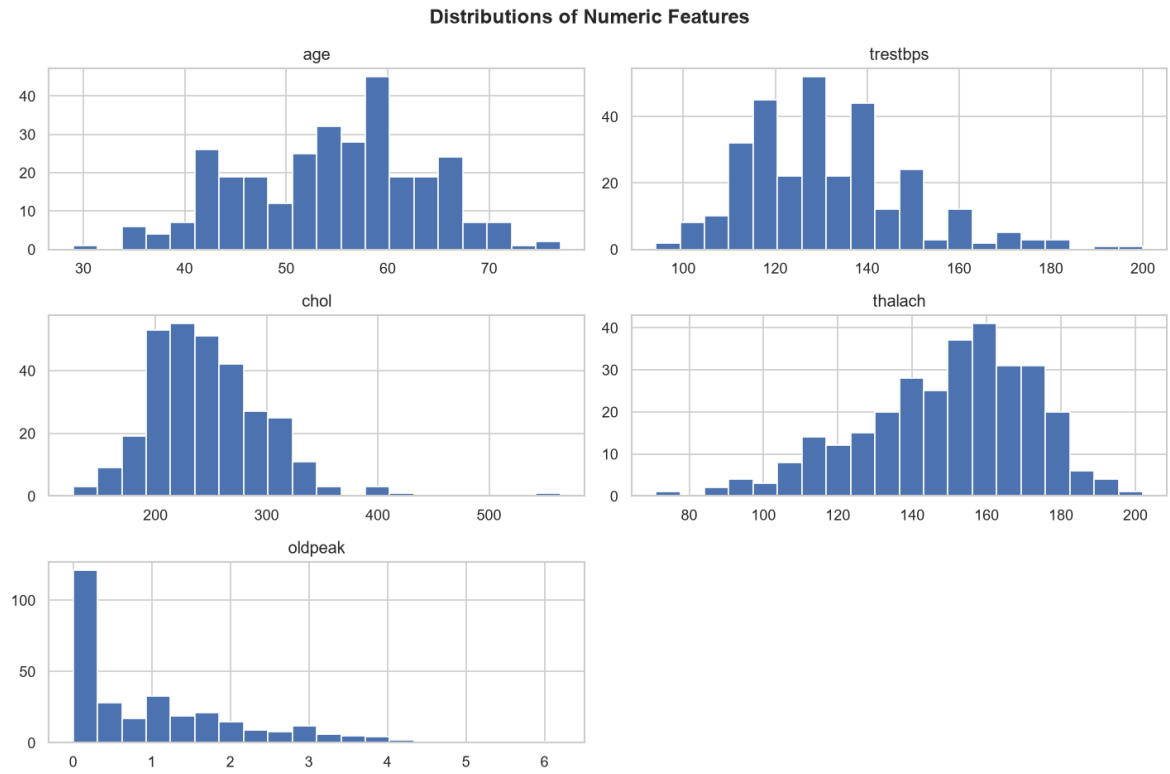


Figure 2. Distributions of numeric features.

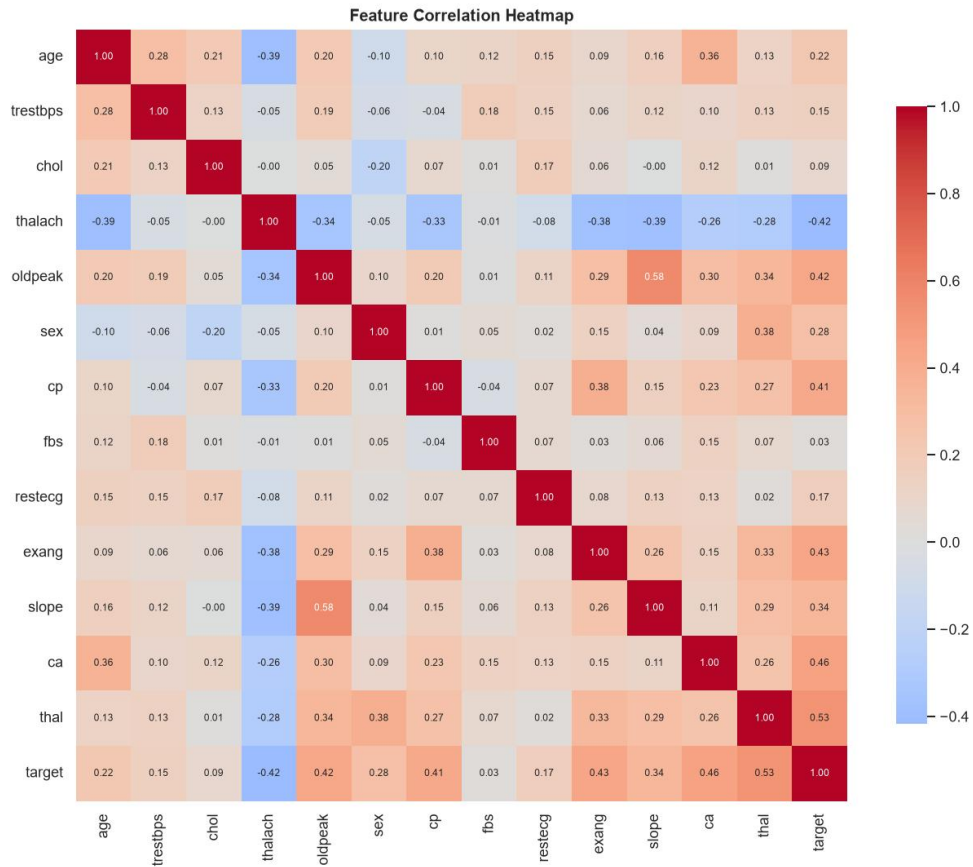


Figure 3. Correlation heatmap. Strongest target correlates: cp, thalach, oldpeak, ca, exang.

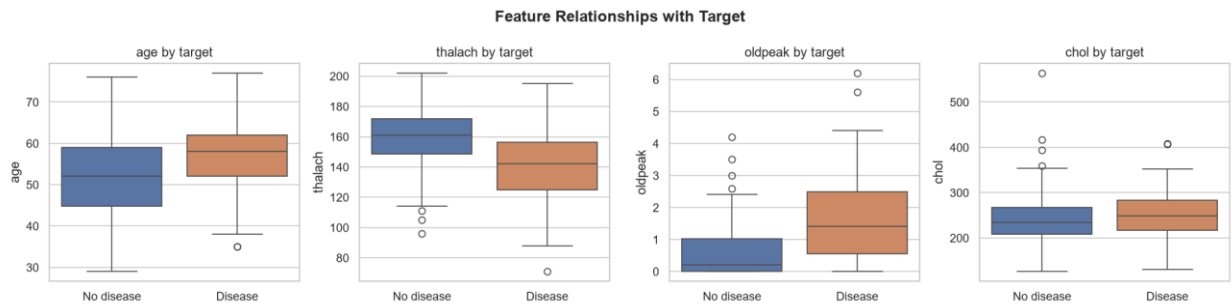


Figure 4. Key features by target - diseased patients show lower max heart rate, higher ST depression, and more affected vessels.

4. Task 2 - Feature Engineering & Model Development

Source files:

- `src/features/pipeline.py` - ColumnTransformer (impute + scale + one-hot)
- `src/models/train.py` - train 3 models, GridSearchCV, select best
- `src/models/evaluate.py` - metrics + ROC / confusion-matrix plots

All feature engineering lives in a single scikit-learn ColumnTransformer (build_preprocessor): numeric features get median imputation + standard scaling; categorical features get most-frequent imputation + one-hot encoding with `handle_unknown='ignore'`. Because the transformer is bundled with the estimator into one Pipeline, it is fitted only on the training folds and reused byte-for-byte at inference, eliminating train/serving skew. Three classifiers are tuned with GridSearchCV (5-fold, ROC-AUC scoring) on a stratified 80/20 split (`random_state=42`).

Commands:

```
python -m src.models.train
```

Model comparison (held-out test set, n=61)

Model	Accuracy	Precision	Recall	F1	Test ROC-AUC	CV ROC-AUC
Logistic Regression *	0.902	0.867	0.929	0.897	0.968	0.898 +/- 0.047
Random Forest	0.885	0.839	0.929	0.881	0.952	0.897 +/- 0.043
XGBoost	0.869	0.813	0.929	0.867	0.947	0.879 +/- 0.025

* Selected production model. Logistic Regression wins on every headline metric, is cheap to serve, and is interpretable (coefficients map to clinical risk factors) - a real advantage in healthcare. Best hyper-parameters: `C=0.1`, `solver=liblinear`. All three models achieve 0.929 recall, the metric we most care about for a screening tool.

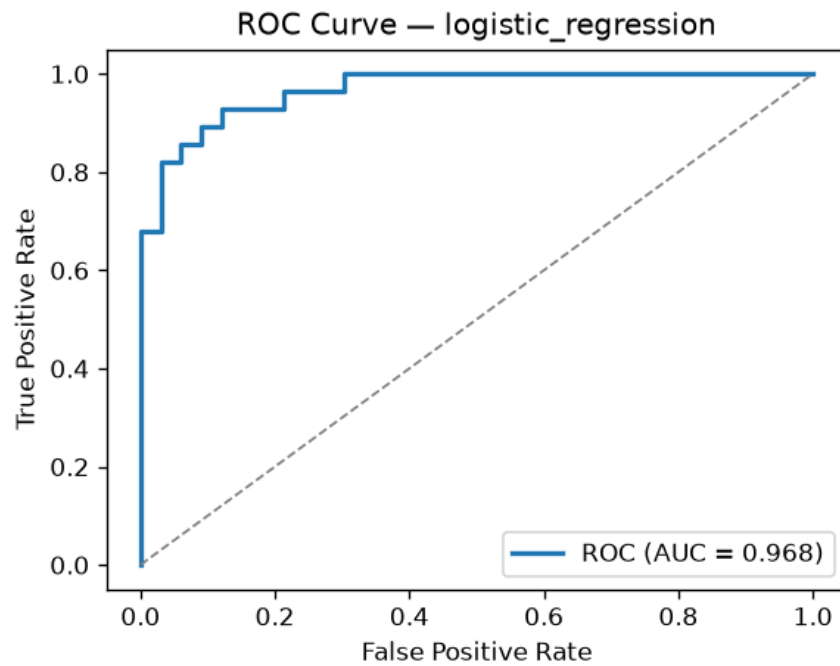


Figure 5. ROC curve - Logistic Regression (AUC = 0.968).

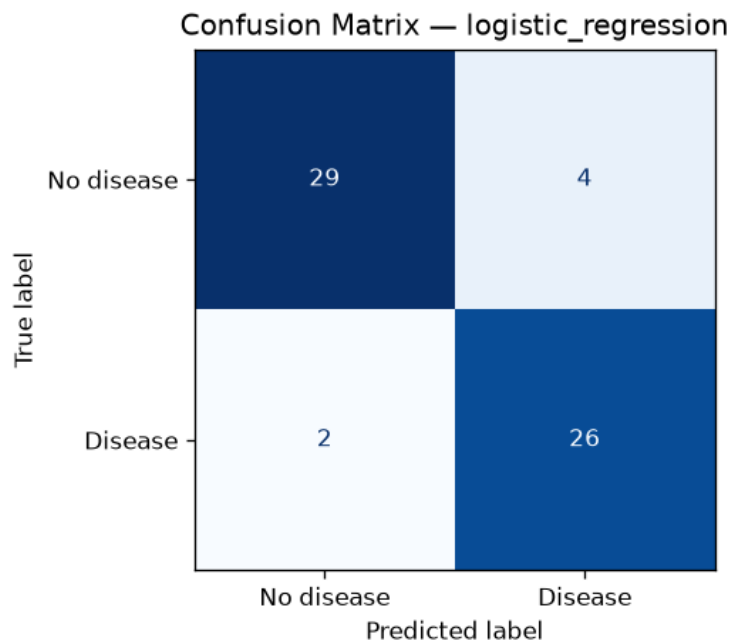


Figure 6. Confusion matrix - Logistic Regression.

5. Task 3 - Experiment Tracking (MLflow)

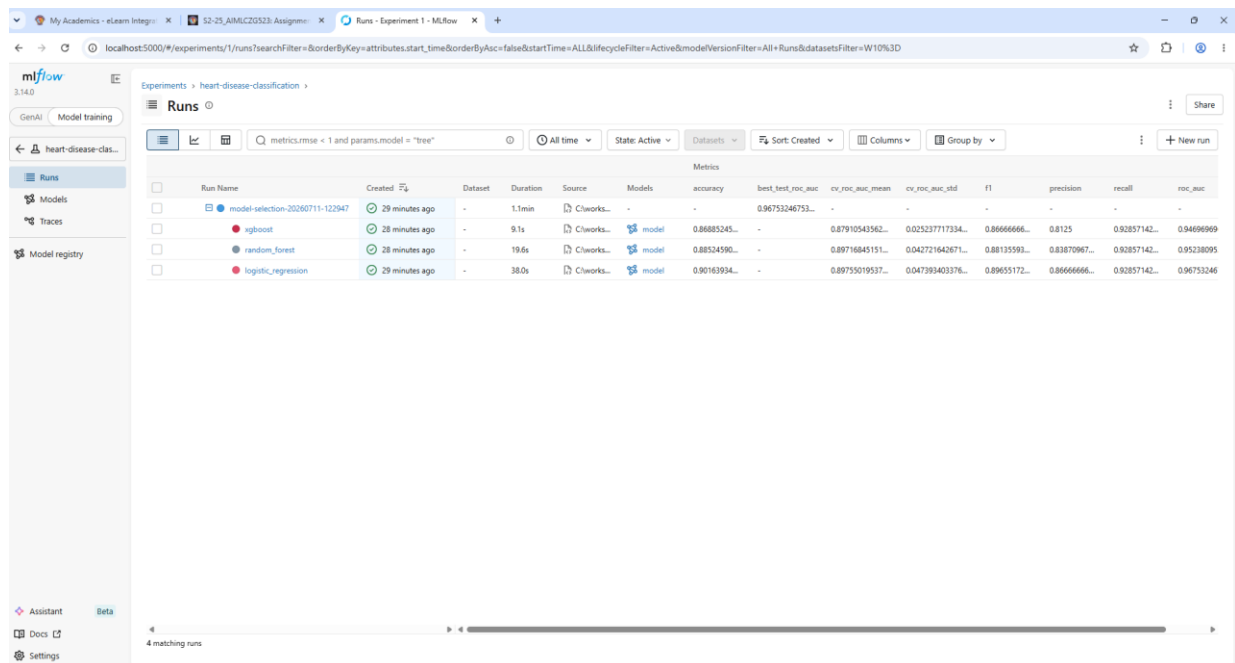
Source files:

- src/models/train.py - mlflow.start_run / log_params / log_metrics / log_artifact / log_model
- src/config.py - MLFLOW_TRACKING_URI (sqlite), MLFLOW_ARTIFACT_LOCATION
- scripts/mlflow_ui.py - launches the UI on Python 3.14 (compat shim)

Training logs everything to MLflow using a SQLite backend (sqlite:///mlflow.db; the file store is deprecated in MLflow 3.x). One parent run (model-selection-*) contains a nested run per model, each logging the best hyper-parameters, all metrics (accuracy, precision, recall, f1, test ROC-AUC, CV ROC-AUC mean/std), the ROC-curve and confusion-matrix artifacts, and the fitted sklearn Pipeline.

Commands:

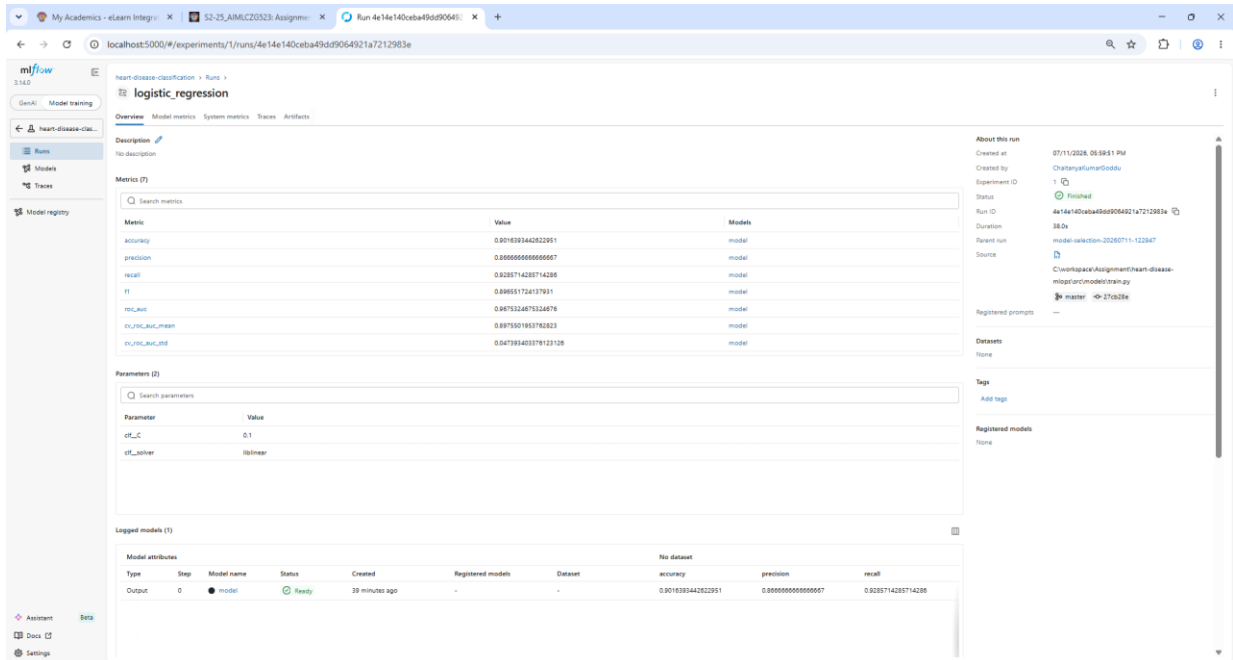
```
python scripts/mlflow_ui.py --port 5000 # then open http://localhost:5000
# (Python <=3.13: mlflow ui --backend-store-uri sqlite:///mlflow.db --port 5000)
```



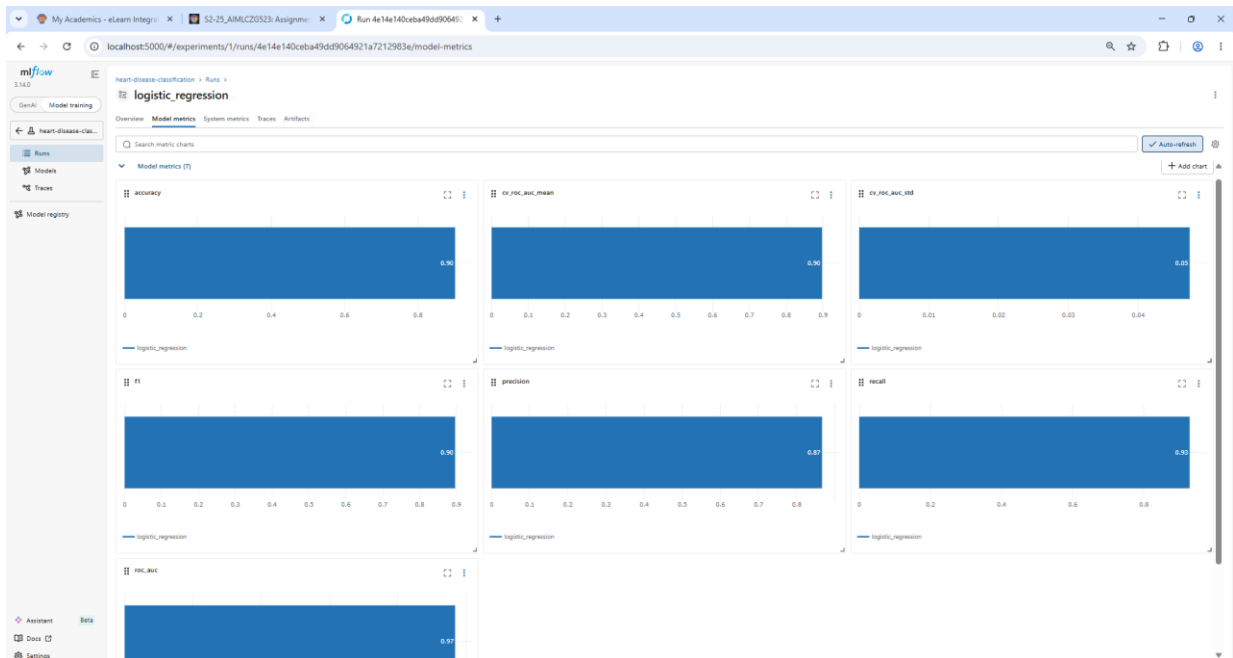
Run Name	Created	Dataset	Duration	Source	Models	accuracy	best_test_roc_auc	cv_roc_auc_mean	cv_roc_auc_std	f1	precision	recall	roc_auc
model-selection-30260711-122947	29 minutes ago	-	1.1min	Clworks...	-	-	0.96753240753...	-	-	-	-	-	-
random_forest	28 minutes ago	-	9.1s	Clworks...	model	0.8885245...	-	0.87910543562...	0.025237717334...	0.86666666...	0.8125	0.82857142...	0.94696969
logistic_regression	28 minutes ago	-	19.6s	Clworks...	model	0.88524590...	-	0.89716845151...	0.042721642671...	0.88135593...	0.83870967...	0.82857142...	0.95238095
logistic_regression	29 minutes ago	-	38.0s	Clworks...	model	0.90163934...	-	0.89755019537...	0.047393403376...	0.89655172...	0.86666666...	0.82857142...	0.96753246

Screenshot: MLflow runs table - all three models compared side by side (accuracy, ROC-AUC, recall, F1, CV scores).

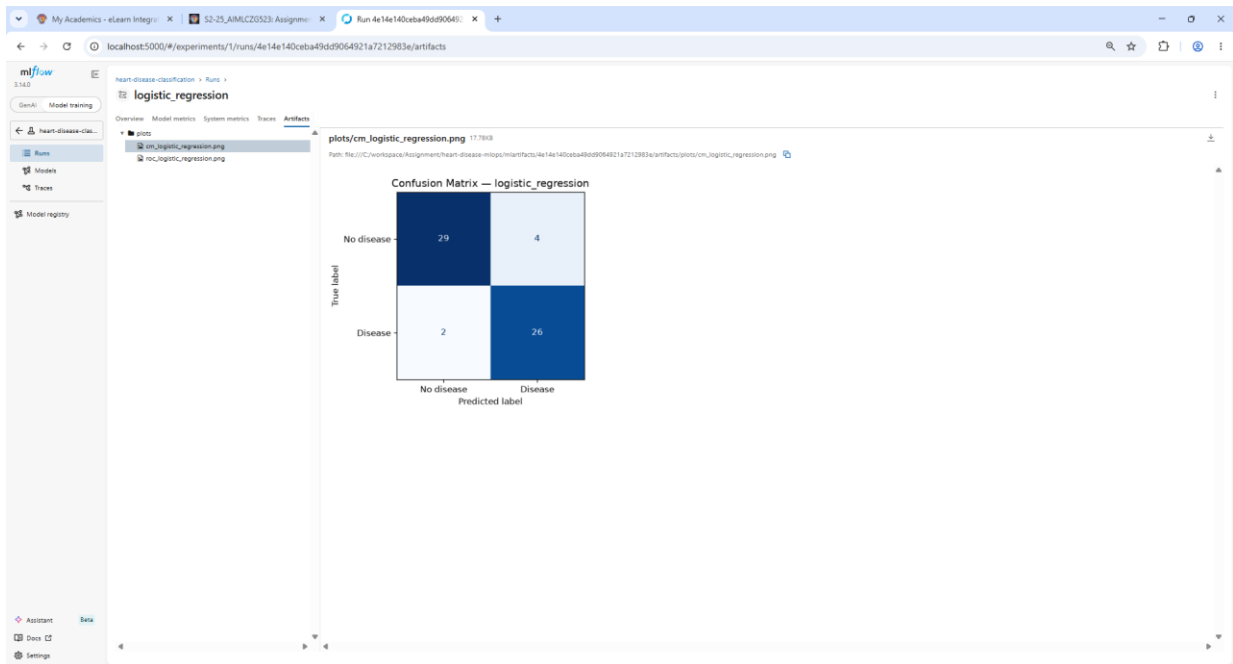
Logistic Regression (selected model)



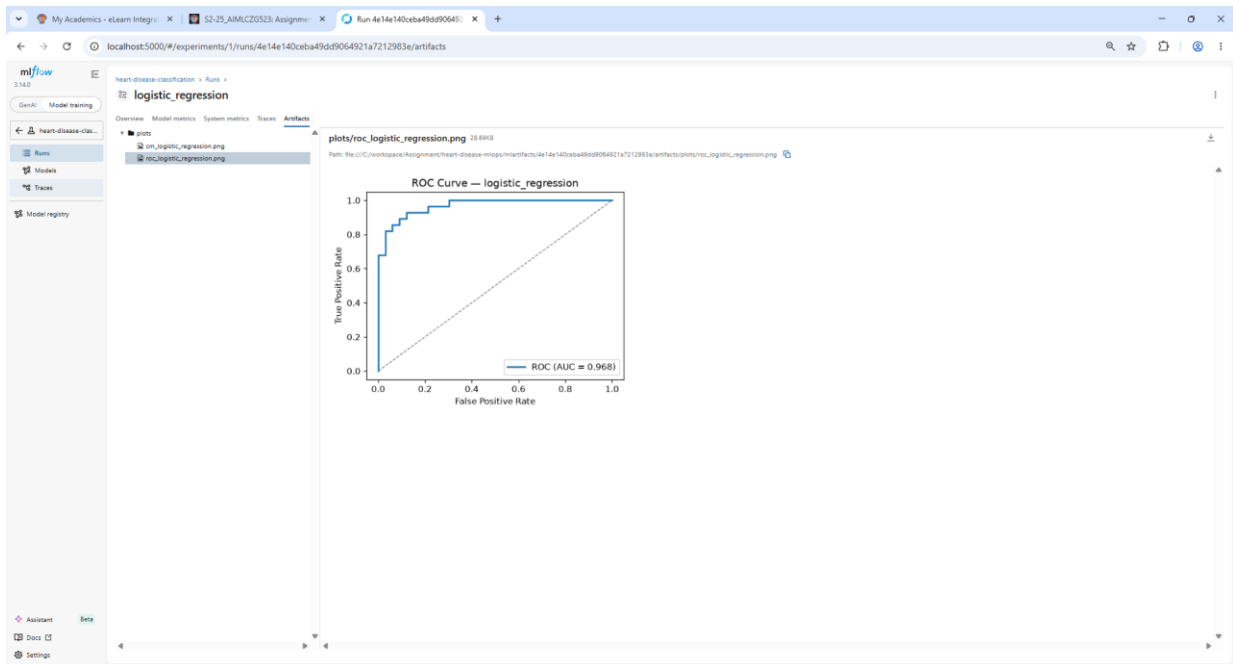
Screenshot: Logistic Regression run overview - metrics + tuned params ($C=0.1$, solver=liblinear) + logged model.



Screenshot: Logistic Regression - model-metric charts.

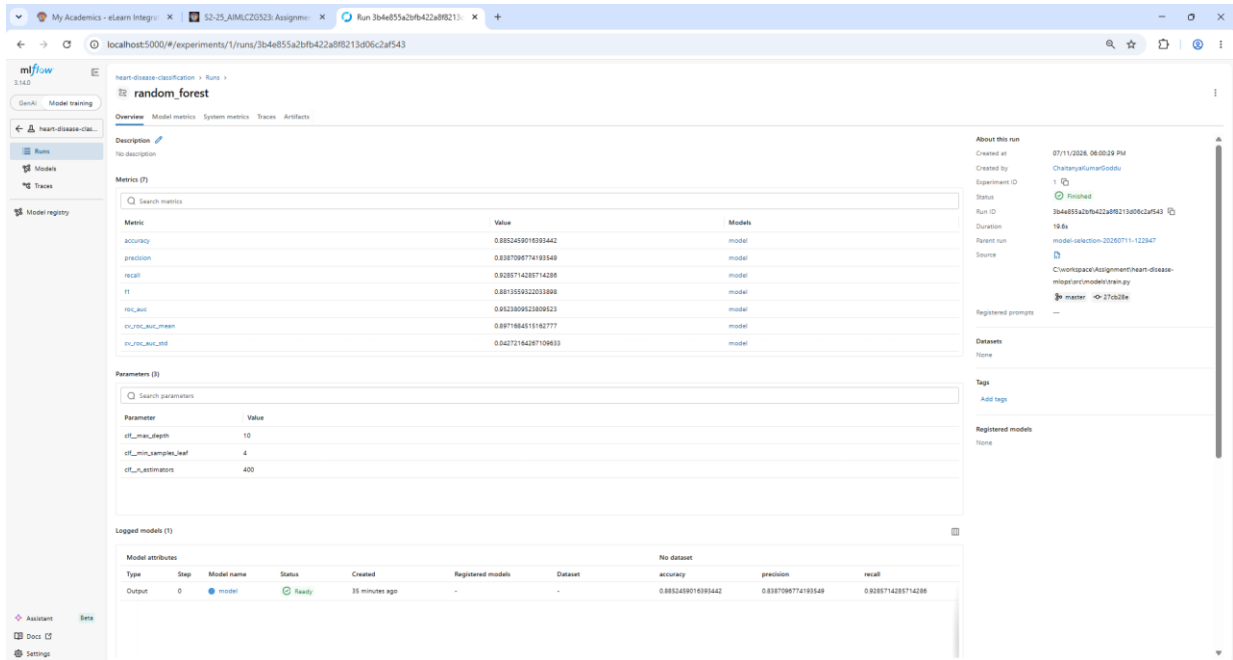


Screenshot: Logistic Regression - logged artifact: confusion matrix.

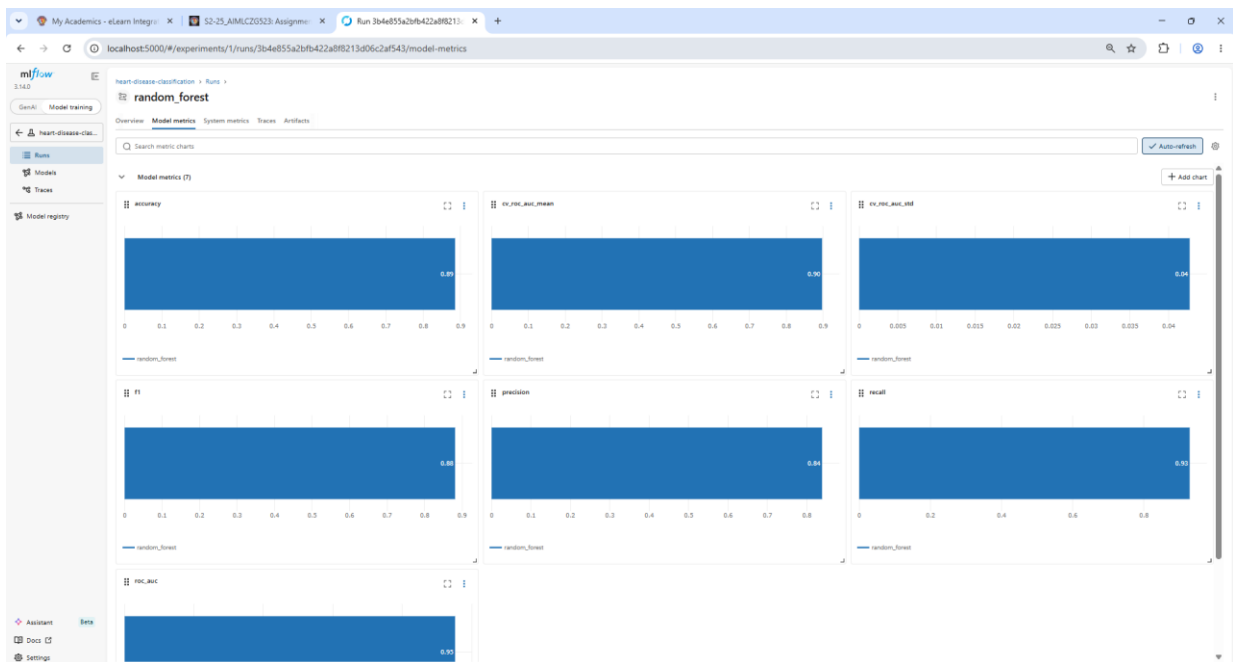


Screenshot: Logistic Regression - logged artifact: ROC curve (AUC = 0.968).

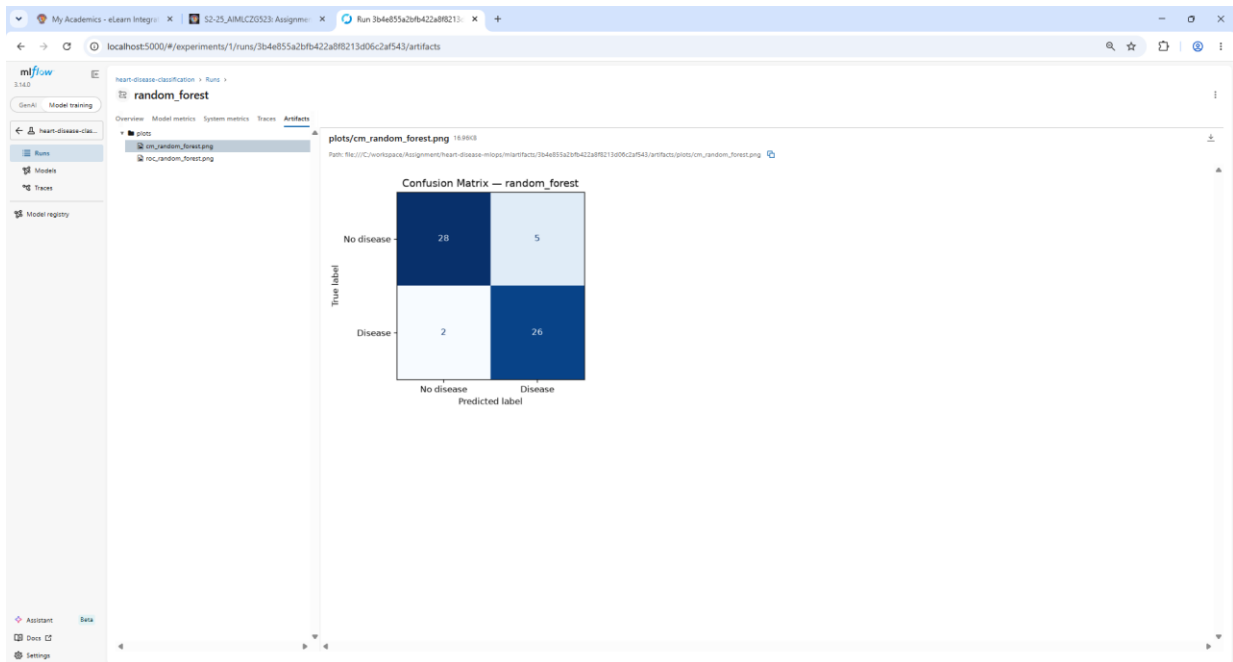
Random Forest



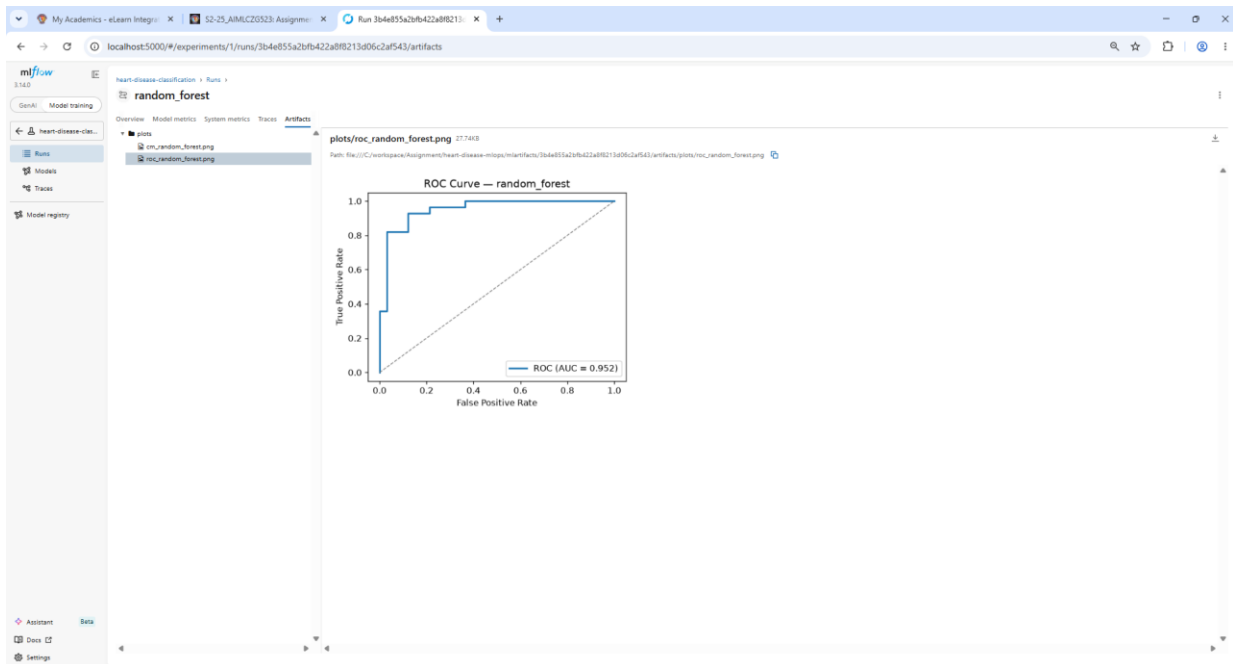
Screenshot: Random Forest run overview - metrics + tuned params ($n_{\text{estimators}}=400$, $\text{max_depth}=10$, $\text{min_samples_leaf}=4$).



Screenshot: Random Forest - model-metric charts.

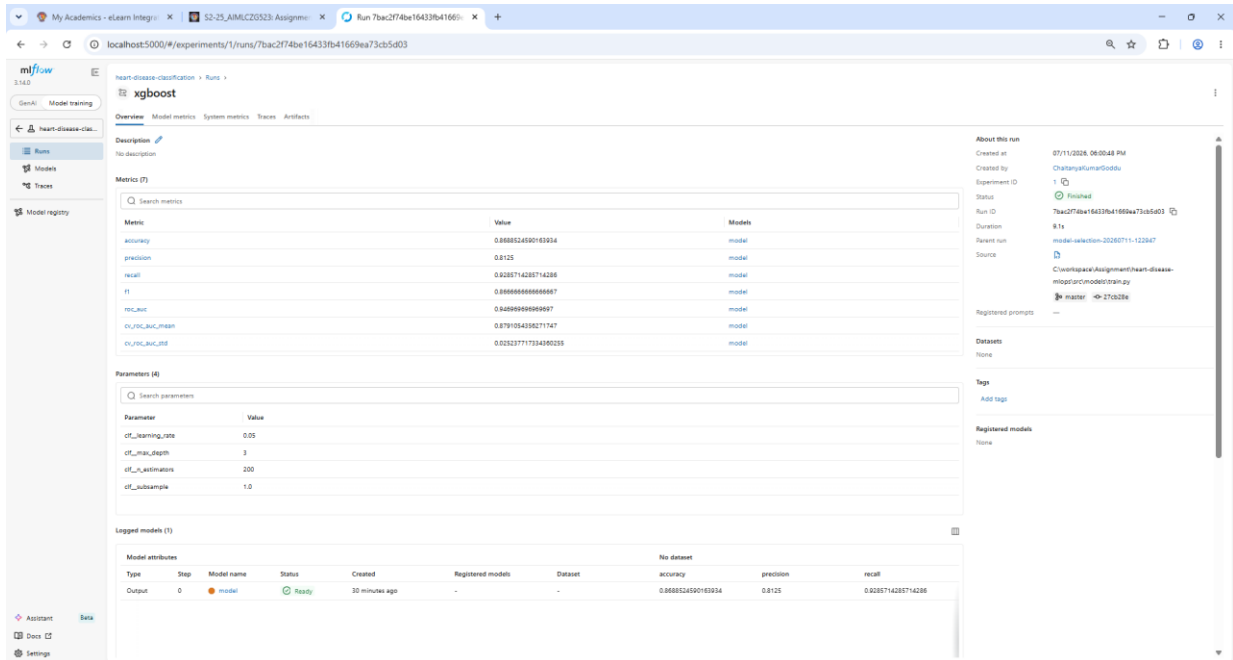


Screenshot: Random Forest - confusion matrix artifact.

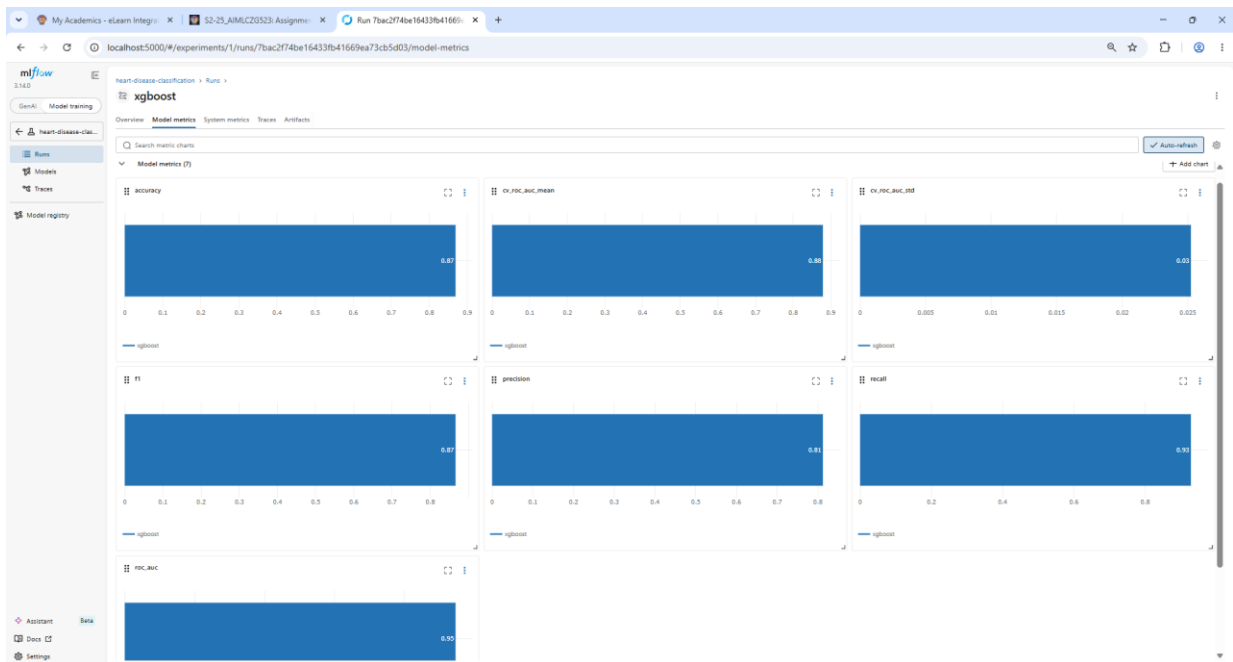


Screenshot: Random Forest - ROC curve artifact (AUC = 0.952).

XGBoost



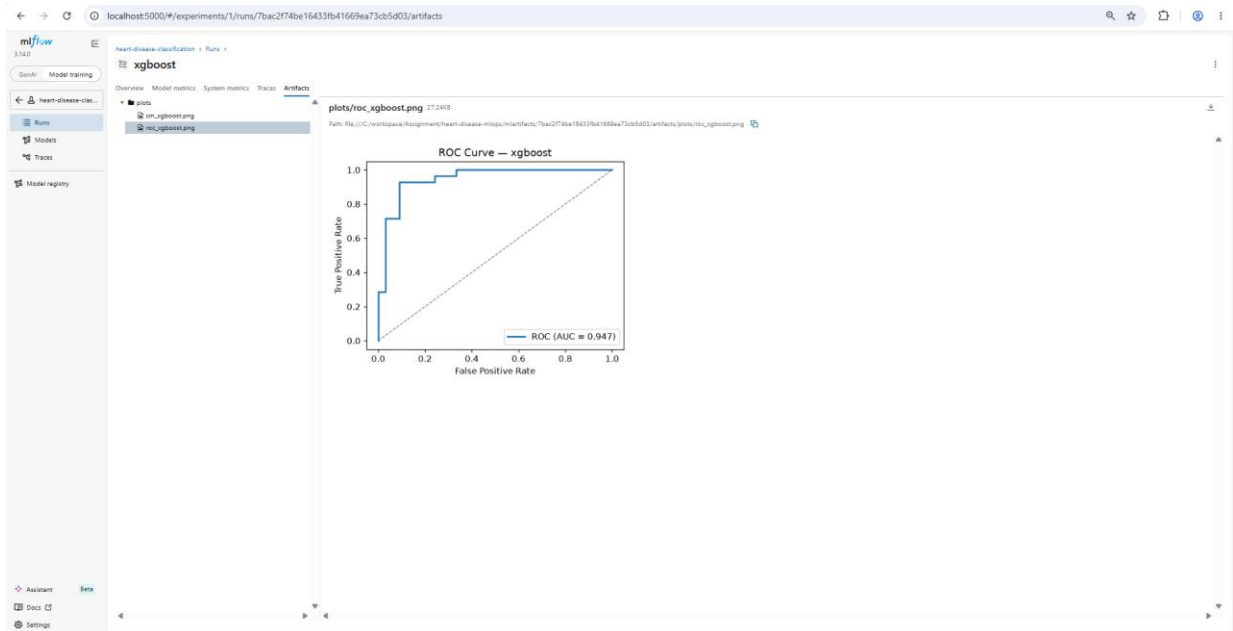
Screenshot: XGBoost run overview - metrics + tuned params (learning_rate=0.05, max_depth=3, n_estimators=200).



Screenshot: XGBoost - model-metric charts.



Screenshot: XGBoost - confusion matrix artifact.



Screenshot: XGBoost - ROC curve artifact (AUC = 0.947).

6. Task 4 - Model Packaging & Reproducibility

Source files:

- `models/model.pkl` - fitted sklearn Pipeline (joblib)
- `models/model_metadata.json` - model name, best params, metrics, feature order
- `src/models/train.py` - `joblib.dump(best_estimator, MODEL_PATH)`
- `requirements.txt` - pinned dependency versions

The winning pipeline is serialised to `models/model.pkl` with joblib, alongside `model_metadata.json` (model name, best params, metrics, feature order, timestamp). Because preprocessing lives inside the pipeline, the artifact is fully self-contained: the API just calls `predict_proba` on a raw feature row. `requirements.txt` pins exact versions and `random_state=42` fixes every stochastic step, so a clean environment reproduces the same model and metrics.

7. Task 5 - CI/CD Pipeline & Automated Testing

Source files:

- tests/conftest.py - offline fixtures (synthetic data + tiny pipeline)
- tests/test_data.py - cleaning / preprocessing tests
- tests/test_model.py - pipeline + metrics tests
- tests/test_api.py - FastAPI /predict, /health, validation, 503, /metrics
- .github/workflows/ci.yml - lint -> test -> train -> docker build + smoke test
- pyproject.toml, .flake8 - pytest / black / isort / flake8 config

The suite has 17 unit/integration tests covering target binarisation, missing-value handling, de-duplication, the column contract, the preprocessor removing NaNs, pipeline output shape/range, metric computation, and the API (happy path, 422 validation errors, 503 when no model, /metrics exposure). The GitHub Actions workflow runs on every push/PR with three dependent jobs - Lint & Test, Train Model, and Docker Build Validation - and fails the run on any lint or test error.

Commands:

```
pytest # 17 passed
flake8 src tests # lint (clean)
black --check src tests && isort --check-only src tests
```

```
File Edit Selection View Go Run Terminal Help
PROBLEMS OUTPUT TERMINAL DEBUG CONSOLE PORTS SONARQUBE

(.venv) PS C:\workspace\Assignment\heart-disease-mlops> pytest
===== test session starts =====
platform win32 -- Python 3.14.0, pytest-9.1.1, pluggy-1.6.0 -- C:\workspace\Assignment\heart-disease-mlops\.venv\Scripts\python.exe
cachedir: .pytest_cache
rootdir: C:\workspace\Assignment\heart-disease-mlops
configfile: pyproject.toml
testpaths: tests
plugins: anyio-4.14.1, cov-7.1.0
collected 17 items

tests/test_api.py::test_health_ok PASSED [ 5%]
tests/test_api.py::test_predict_returns_valid_schema PASSED [ 11%]
tests/test_api.py::test_predict_rejects_out_of_range PASSED [ 17%]
tests/test_api.py::test_predict_rejects_missing_field PASSED [ 23%]
tests/test_api.py::test_predict_503_when_model_missing PASSED [ 29%]
tests/test_api.py::test_metrics_endpoint_exposed PASSED [ 35%]
tests/test_data.py::test_clean_binaries_target PASSED [ 41%]
tests/test_data.py::test_clean_coerces_missing_markers_to_nan PASSED [ 47%]
tests/test_data.py::test_clean_drops_duplicates PASSED [ 52%]
tests/test_data.py::test_clean_column_contract PASSED [ 58%]
tests/test_data.py::test_clean_numeric_dtypes PASSED [ 64%]
tests/test_model.py::test_preprocessor_removes_nans PASSED [ 70%]
tests/test_model.py::test_preprocessor_output_row_count PASSED [ 76%]
tests/test_model.py::test_pipeline_predicts_binary PASSED [ 82%]
tests/test_model.py::test_pipeline_proba_in_range PASSED [ 88%]
tests/test_model.py::test_compute_metrics_keys PASSED [ 94%]
tests/test_model.py::test_pipeline_is_reusable_single_row PASSED [100%]

===== tests coverage =====
coverage: platform win32, python 3.14.0-final-0

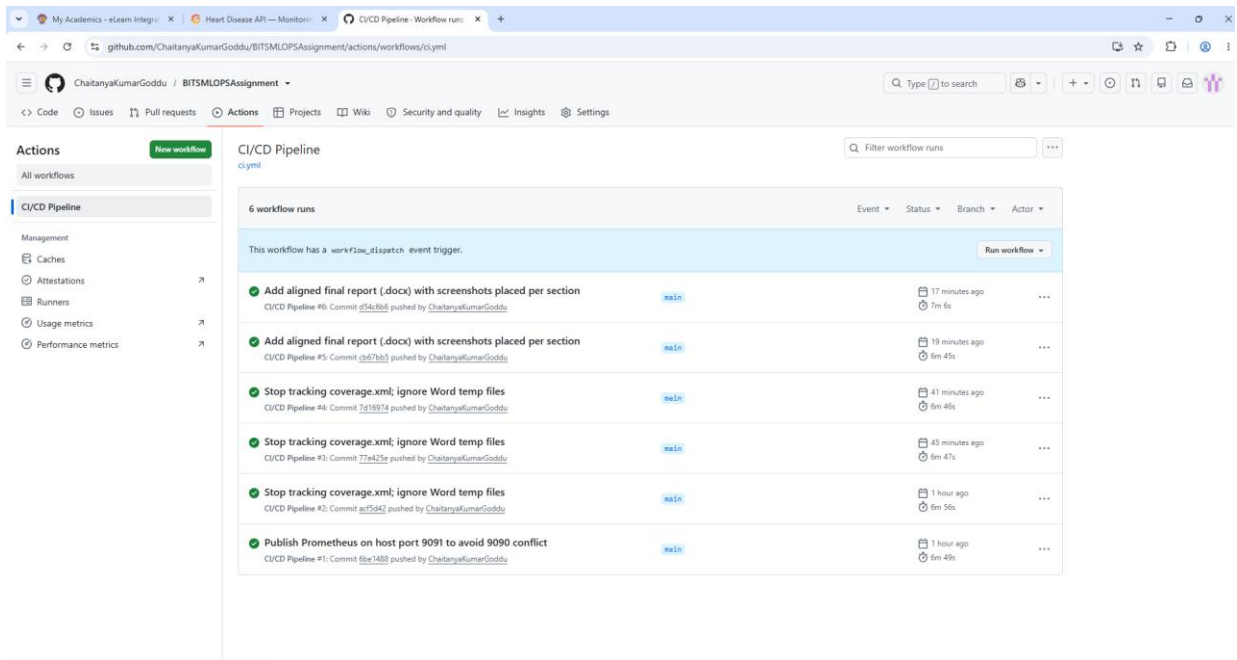
Name                               Stmts  Miss  Cover   Missing
-----
src\__init__.py                      1      0  100%
src\api\__init__.py                  0      0  100%
src\api\main.py                     61     12   80%   61-68, 74-76, 108
src\api\schemas.py                 27      0  100%
src\config.py                       27      2   93%   85-92
src\data\__init__.py                 0      0  100%
src\data\veda.py                    70     70    0%   11-131
src\data\preprocess.py              25     10   60%   60-64, 69-76, 80
src\features\__init__.py             0      0  100%
src\features\pipeline.py            11      0  100%
src\models\__init__.py              0      0  100%
src\models\evaluate.py              32     22   31%   35-40, 54-67
src\models\train.py                 83     83    0%   21-245
src\utils\__init__.py               0      0  100%
src\utils\architecture.py           48     48    0%   9-157
-----
TOTAL                               385    247   36%
Coverage XML written to file coverage.xml

===== 17 passed in 1.85s =====
(.venv) PS C:\workspace\Assignment\heart-disease-mlops> []
```

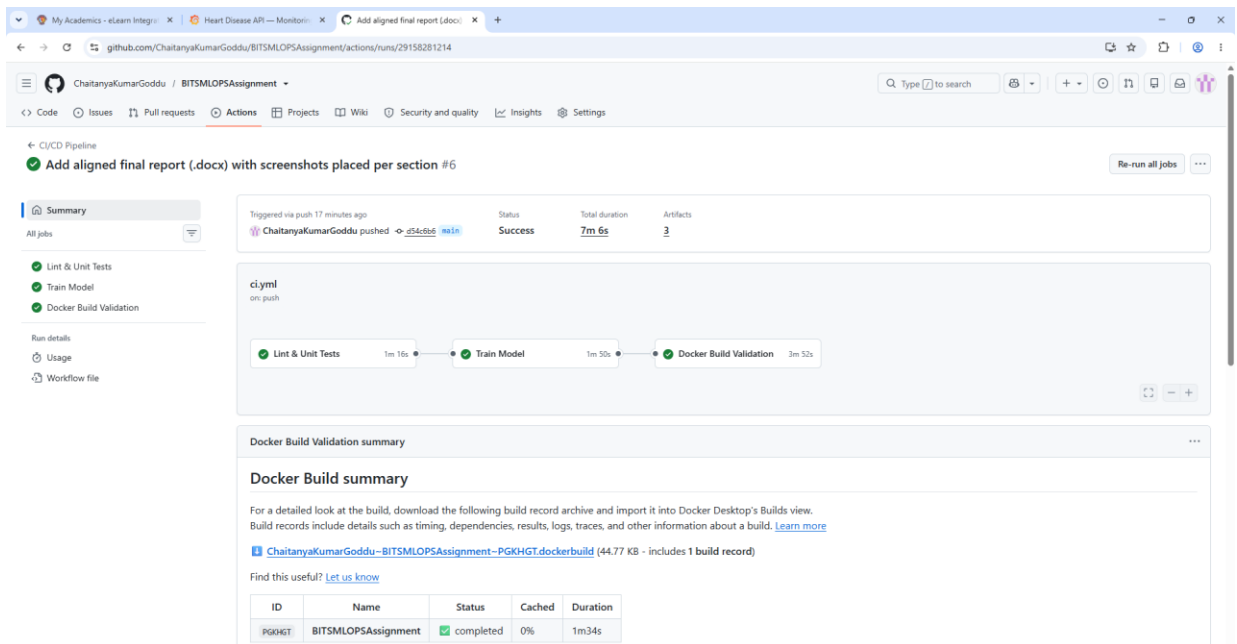
Screenshot: Local Pytest run - all 17 tests pass, with the coverage summary.

GitHub Actions CI/CD pipeline

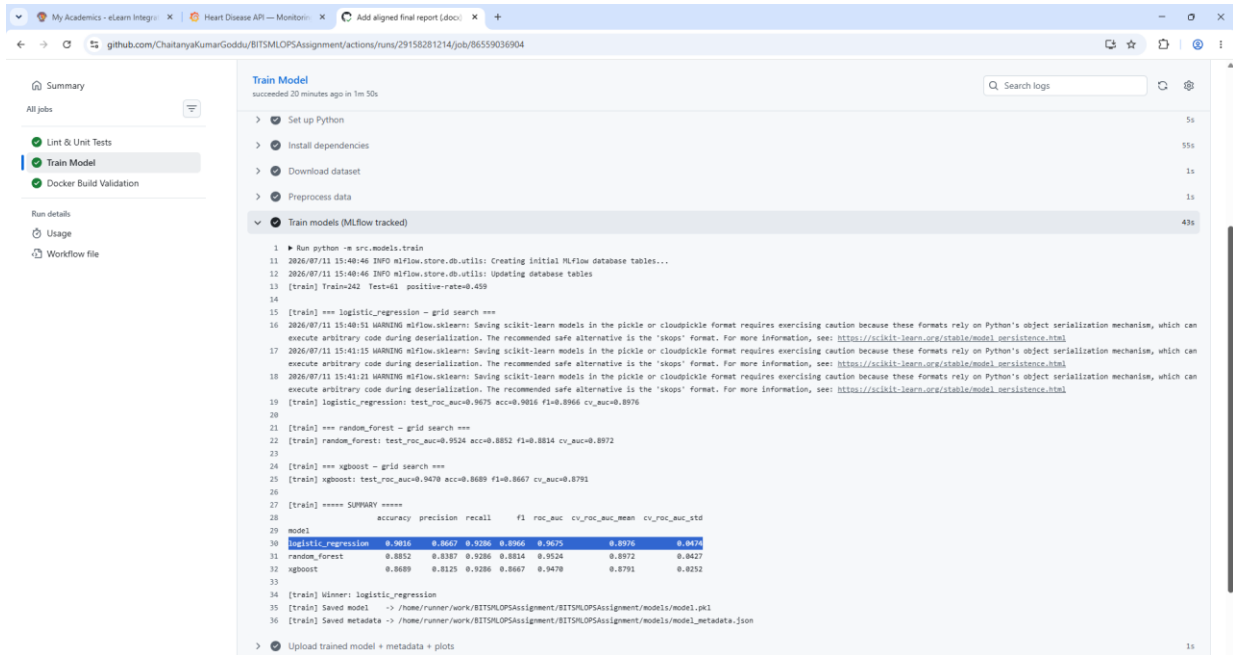
The GitHub Actions CI/CD pipeline runs automatically on every push to main and has passed consistently (6 of 6 runs green). Each run executes the three dependent jobs in sequence and fails fast on any lint or test error, so broken code never reaches the container-build stage.



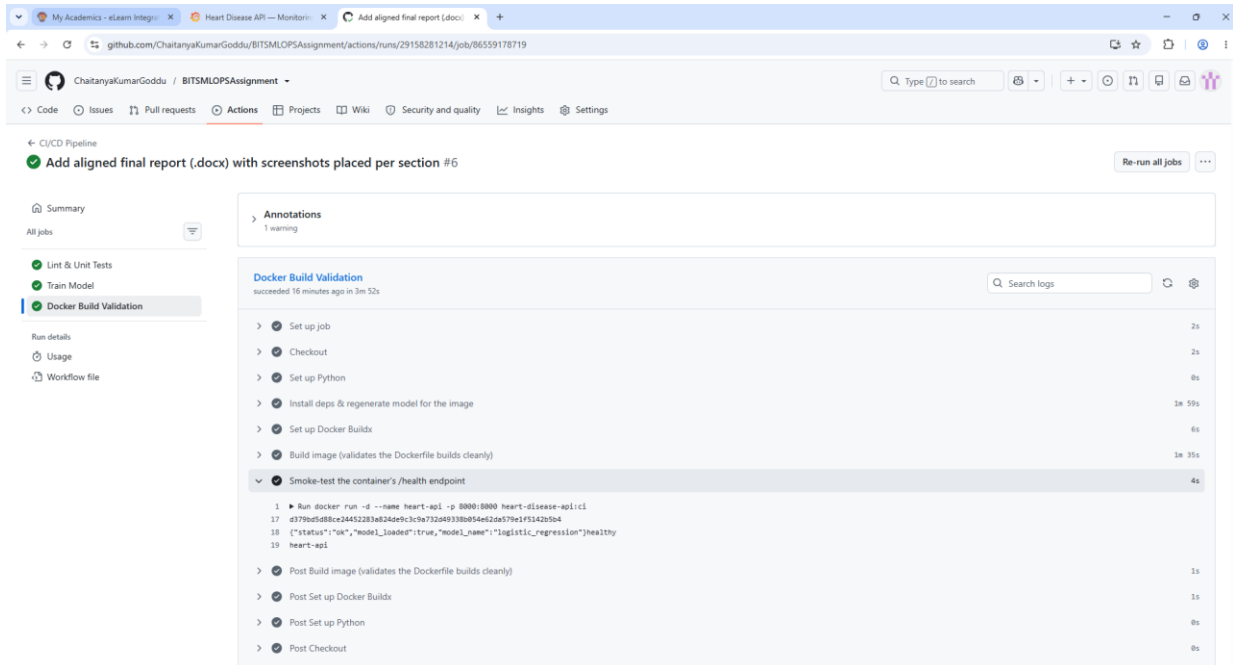
Screenshot: GitHub Actions - the CI/CD Pipeline triggers on every push; all runs completed successfully (green).



Screenshot: CI/CD run summary - the three jobs run in sequence: Lint & Unit Tests -> Train Model -> Docker Build Validation (Success, 3 artifacts).



Screenshot: Train Model job log - GridSearchCV comparison table; 'Winner: logistic_regression'; model + metadata saved.



Screenshot: Docker Build Validation job - the container /health smoke-test returns {"status":"ok","model_loaded":true,"model_name":"logistic_regression"}.

My Academics - eLearn Integr...

Heart Disease API - Monitor...

Add aligned final report (docu...

github.com/ChaitanyaKumarGodd...BITSMLOPSAssignment/actions/runs/29158281214

Summary

All jobs

Lint & Unit Tests

Train Model

Docker Build Validation

Run details

Usage

Workflow file

Build inputs

context: .
load: true
tags:
- heart-disease-api:cl

Job summary generated at run-time

Annotations

3 warnings

Lint & Unit Tests

Node.js 20 is deprecated. The following actions target Node.js 20 but are being forced to run on Node.js 24: actions/checkout@v4, actions/setup-python@v5, actions/upload-artifact@v4. For more information see: [https://gl...](#)
Show more

Train Model

Node.js 20 is deprecated. The following actions target Node.js 20 but are being forced to run on Node.js 24: actions/checkout@v4, actions/setup-python@v5, actions/upload-artifact@v4. For more information see: [https://gl...](#)
Show more

Docker Build Validation

Node.js 20 is deprecated. The following actions target Node.js 20 but are being forced to run on Node.js 24: actions/checkout@v4, actions/setup-python@v5, docker/build-push-action@v6, docker/setup-buildx-action@v3. For ..
Show more

Artifacts

Produced during runtime

Name	Size	Digest		
ChaitanyaKumarGodd...BITSMLOPSAssignment-PGKHGT.dockerbuil	44.8 KB	sha256:48793246024658bca1d61ed37d0b75653ba0ac0621b8f84ba8ad5c0b79...		
test-coverage	1.73 KB	sha256:58ca53bd48796c0681d6c3f8817b50d8a4784558cdeb3b0508a1190ba034...		
trained-model	455 KB	sha256:38e4c61678decffe2897f424a8efcd377eac31ae52f3342759242a8557a5b...		

Screenshot: CI/CD run artifacts - trained-model, test-coverage, and the Docker build record uploaded for each run.

8. Task 6 - Containerisation (Docker) & Serving API

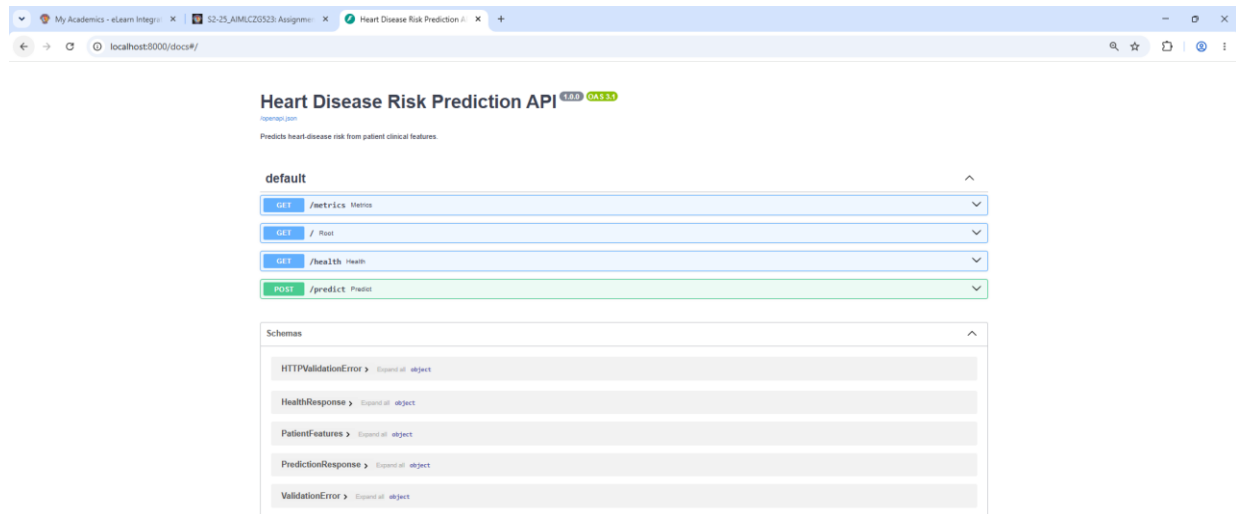
Source files:

- Dockerfile - multi-stage build, non-root user, HEALTHCHECK
- .dockerignore - keep the image lean
- src/api/main.py - FastAPI app: /predict, /health, /metrics
- src/api/schemas.py - Pydantic request/response models + validation
- sample_request.json - example high-risk patient payload

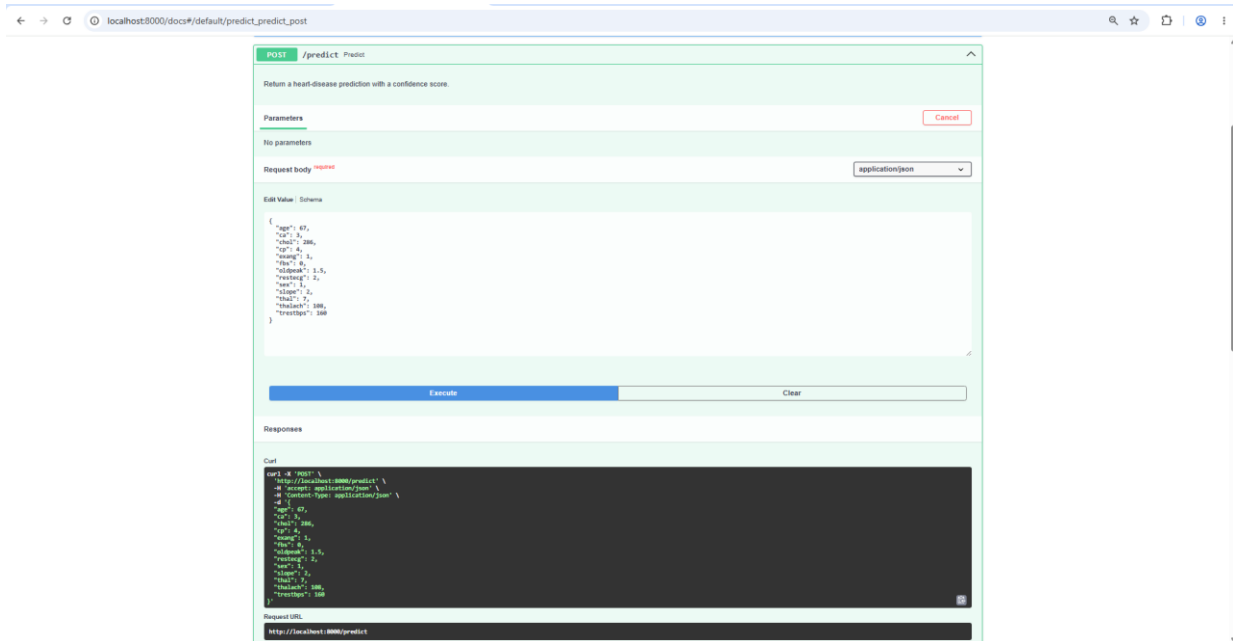
A multi-stage Dockerfile (Python 3.12-slim) builds wheels in stage 1 and installs them offline in stage 2, runs as a non-root user, exposes port 8000, and defines a container HEALTHCHECK against /health. The /predict endpoint accepts JSON, validates it with Pydantic (out-of-range values are rejected with HTTP 422), and returns the prediction plus a confidence score. Swagger UI is auto-generated at /docs.

Commands:

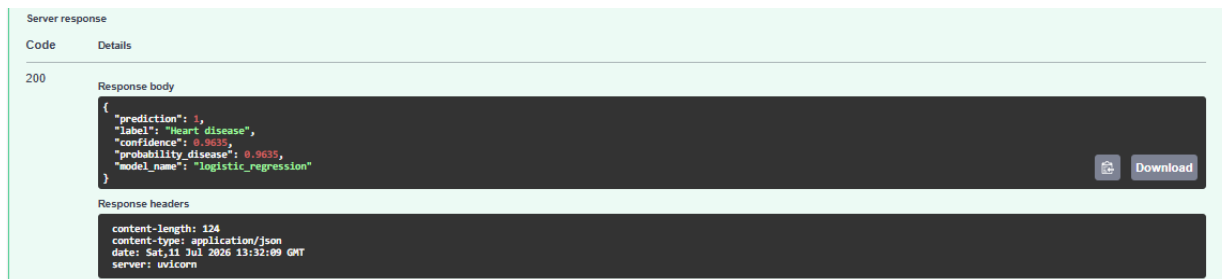
```
docker build -t heart-disease-api:latest .  
docker run --rm -p 8000:8000 heart-disease-api:latest  
curl.exe -X POST http://localhost:8000/predict -H "Content-Type:  
application/json" -d "@sample_request.json"
```



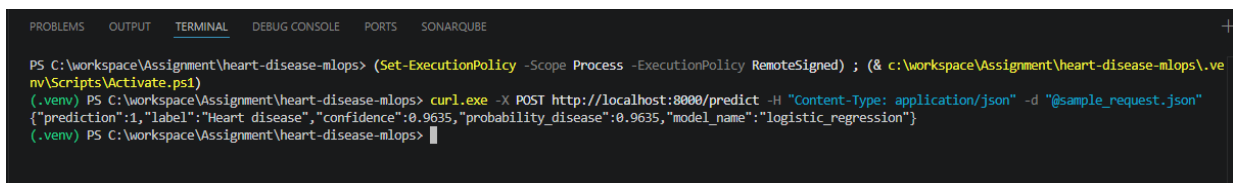
Screenshot: FastAPI Swagger UI (/docs) exposing /predict, /health, /metrics and / endpoints.



Screenshot: Swagger UI - /predict 'Try it out' with the sample request body.



Screenshot: Swagger UI - /predict returns HTTP 200: prediction=1, confidence 0.9635.



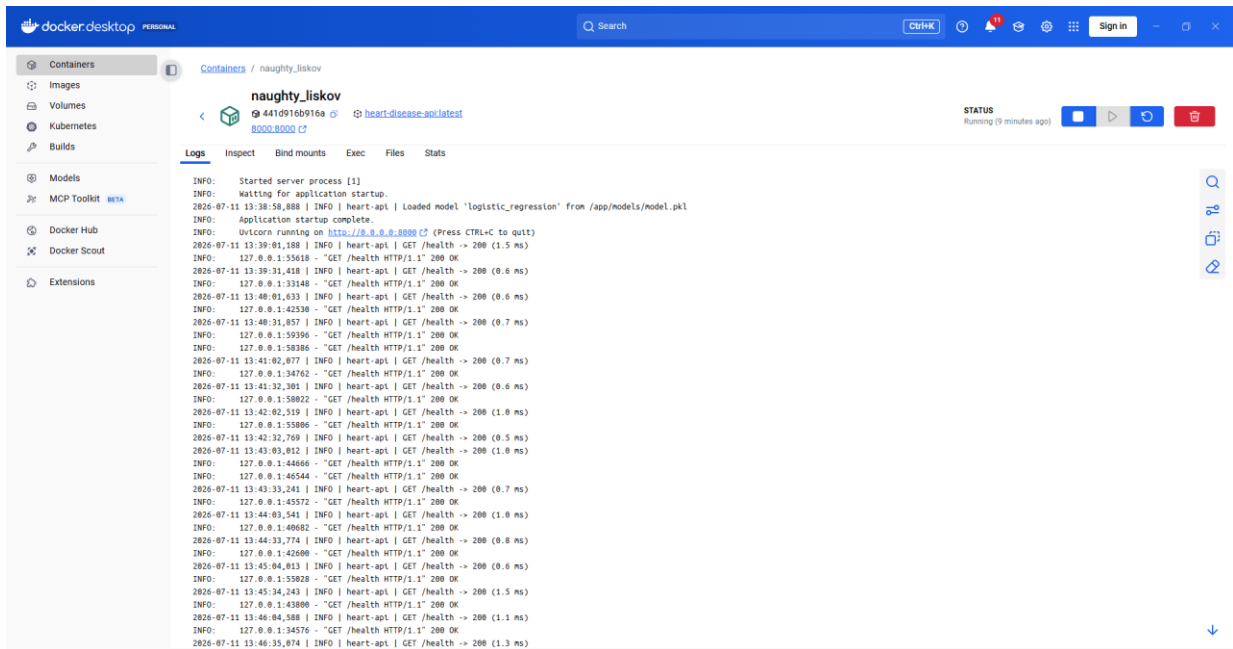
Screenshot: Local /predict call via curl returning prediction + confidence.

```
PROBLEMS OUTPUT TERMINAL DEBUG CONSOLE PORTS SONARQUBE
PS C:\workspace\Assignment\heart-disease-mlops> (Set-ExecutionPolicy -Scope Process -ExecutionPolicy RemoteSigned) ; (& c:\workspace\Assignment\heart-disease-mlops\venv\Scripts\Activate.ps1)
(.venv) PS C:\workspace\Assignment\heart-disease-mlops> curl.exe -X POST http://localhost:8000/predict -H "Content-Type: application/json" -d "@sample_request.json"
{"prediction":1,"label":"Heart disease","confidence":0.9635,"probability_disease":0.9635,"model_name":"logistic_regression"}
(.venv) PS C:\workspace\Assignment\heart-disease-mlops> docker build -t heart-disease-api:latest .
[+] Building 87.4s (18/18) FINISHED
=> [internal] load build definition from Dockerfile
=> => transferring dockerfile: 1.54kB
=> resolve image config for docker-image://docker.io/docker/dockerfile:1
=> CACHED docker-image://docker.io/docker/dockerfile:1@sha256:87999aa3d42bdc6bea60565083ee17e86d1f3339802f543c0d03998580f9cb89
=> => resolve docker.io/docker/dockerfile:1@sha256:87999aa3d42bdc6bea60565083ee17e86d1f3339802f543c0d03998580f9cb89
=> [internal] load metadata for docker.io/library/python:3.12-slim
=> [internal] load .dockerignore
=> => transferring context: 224B
=> [builder 1/4] FROM docker.io/library/python:3.12-slim@sha256:423ed6ab25b1921a477529254bfeeabf5855151dc2c3141699a1bfc852199fbf
=> => resolve docker.io/library/python:3.12-slim@sha256:423ed6ab25b1921a477529254bfeeabf5855151dc2c3141699a1bfc852199fbf
=> [internal] load build context
=> => transferring context: 31.55kB
=> CACHED [runtime 2/8] RUN useradd --create-home --uid 1000 appuser
=> CACHED [runtime 3/8] WORKDIR /app
=> CACHED [builder 2/4] WORKDIR /build
=> CACHED [builder 3/4] COPY requirements.txt .
=> CACHED [builder 4/4] RUN pip wheel --wheel-dir=/wheels -r requirements.txt
=> CACHED [runtime 4/8] COPY --from-builder /wheels /wheels
=> CACHED [runtime 5/8] COPY requirements.txt .
=> [runtime 6/8] RUN pip install --no-cache-dir --no-index --find-links=/wheels -r requirements.txt && rm -rf /wheels
=> [runtime 7/8] COPY src/ ./src/
=> [runtime 8/8] COPY models/ ./models/
=> exporting to image
=> => exporting layers
=> => exporting manifest sha256:ce41596b7e0c7309ff1a2d21a1bc1fd997aa401516d42c66a766e2c6aa2500c3
=> => exporting config sha256:d1f900fbcfc8b8db1497b87b49da8e978584650f3047b8849ded5b1c7bed3ee
=> => exporting attestation manifest sha256:7a04e357fd881a1b02e4e5c1911c55fa50b703774dcd8ba7f671aceb10297e7
=> => exporting manifest list sha256:8d2c6fbf646775b699a2e43aa27908b138cb13f24de6b499ab415964a5958174
=> => naming to docker.io/library/heart-disease-api:latest
=> => unpacking to docker.io/library/heart-disease-api:latest
```

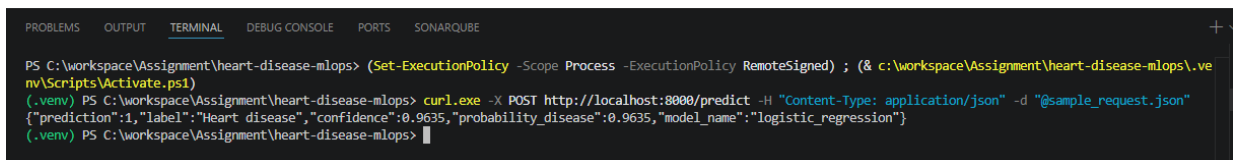
Screenshot: Docker image build - multi-stage build completes (18/18 FINISHED).

```
(.venv) PS C:\workspace\Assignment\heart-disease-mlops> docker run --rm -p 8000:8000 heart-disease-api:latest
INFO: Started server process [1]
INFO: Waiting for application startup.
2026-07-11 13:38:58,888 | INFO | heart-api | Loaded model 'logistic_regression' from /app/models/model.pkl
INFO: Application startup complete.
INFO: Uvicorn running on http://0.0.0.0:8000 (Press CTRL+C to quit)
INFO: 127.0.0.1:55618 - "GET /health HTTP/1.1" 200 OK
2026-07-11 13:39:01,188 | INFO | heart-api | GET /health -> 200 (1.5 ms)
INFO: 127.0.0.1:33148 - "GET /health HTTP/1.1" 200 OK
2026-07-11 13:39:31,418 | INFO | heart-api | GET /health -> 200 (0.6 ms)
2026-07-11 13:40:01,633 | INFO | heart-api | GET /health -> 200 (0.6 ms)
INFO: 127.0.0.1:42530 - "GET /health HTTP/1.1" 200 OK
2026-07-11 13:40:31,857 | INFO | heart-api | GET /health -> 200 (0.7 ms)
INFO: 127.0.0.1:59396 - "GET /health HTTP/1.1" 200 OK
2026-07-11 13:41:02,077 | INFO | heart-api | GET /health -> 200 (0.7 ms)
INFO: 127.0.0.1:58386 - "GET /health HTTP/1.1" 200 OK
2026-07-11 13:41:32,301 | INFO | heart-api | GET /health -> 200 (0.6 ms)
INFO: 127.0.0.1:34762 - "GET /health HTTP/1.1" 200 OK
INFO: 127.0.0.1:58022 - "GET /health HTTP/1.1" 200 OK
2026-07-11 13:42:02,519 | INFO | heart-api | GET /health -> 200 (1.0 ms)
2026-07-11 13:42:32,769 | INFO | heart-api | GET /health -> 200 (0.5 ms)
INFO: 127.0.0.1:55806 - "GET /health HTTP/1.1" 200 OK
2026-07-11 13:43:03,012 | INFO | heart-api | GET /health -> 200 (1.0 ms)
INFO: 127.0.0.1:44666 - "GET /health HTTP/1.1" 200 OK
2026-07-11 13:43:33,241 | INFO | heart-api | GET /health -> 200 (0.7 ms)
INFO: 127.0.0.1:46544 - "GET /health HTTP/1.1" 200 OK
2026-07-11 13:44:03,541 | INFO | heart-api | GET /health -> 200 (1.0 ms)
INFO: 127.0.0.1:45572 - "GET /health HTTP/1.1" 200 OK
2026-07-11 13:44:33,774 | INFO | heart-api | GET /health -> 200 (0.8 ms)
INFO: 127.0.0.1:40682 - "GET /health HTTP/1.1" 200 OK
2026-07-11 13:45:04,013 | INFO | heart-api | GET /health -> 200 (0.6 ms)
INFO: 127.0.0.1:42600 - "GET /health HTTP/1.1" 200 OK
2026-07-11 13:45:34,243 | INFO | heart-api | GET /health -> 200 (1.5 ms)
INFO: 127.0.0.1:55028 - "GET /health HTTP/1.1" 200 OK
INFO: 127.0.0.1:43800 - "GET /health HTTP/1.1" 200 OK
2026-07-11 13:46:04,588 | INFO | heart-api | GET /health -> 200 (1.1 ms)
INFO: 127.0.0.1:34576 - "GET /health HTTP/1.1" 200 OK
2026-07-11 13:46:35,074 | INFO | heart-api | GET /health -> 200 (1.3 ms)
INFO: 127.0.0.1:39746 - "GET /health HTTP/1.1" 200 OK
2026-07-11 13:47:05,812 | INFO | heart-api | GET /health -> 200 (0.8 ms)
```

Screenshot: Container startup logs - model 'logistic_regression' loaded, health checks returning 200.



Screenshot: Docker Desktop - the heart-disease-api container running on port 8000.



Screenshot: /predict verified against the running container (prediction=1, confidence 0.9635).

9. Task 7 - Production Deployment (Kubernetes)

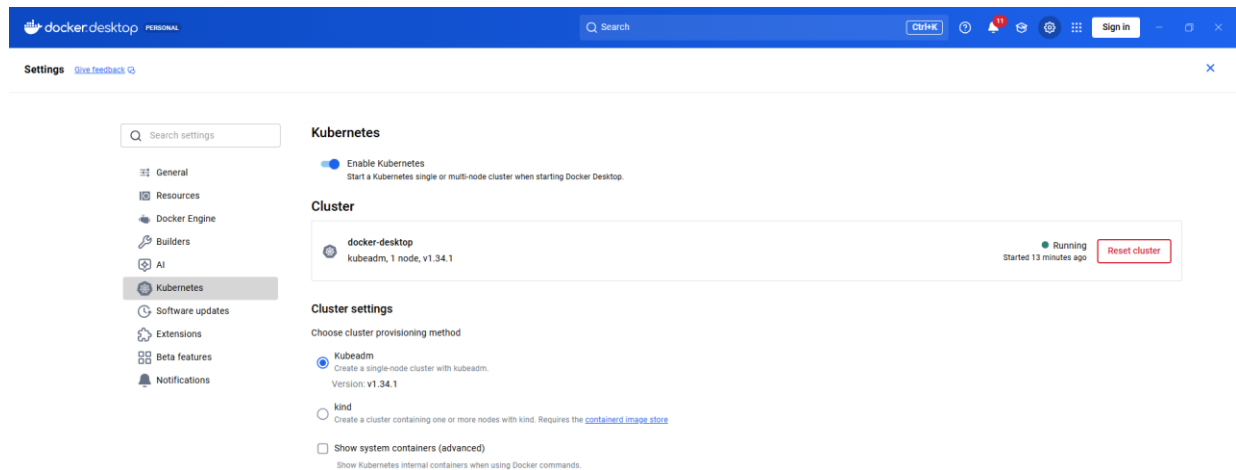
Source files:

- k8s/deployment.yaml - 2 replicas, resource limits, readiness/liveness probes
- k8s/service.yaml - LoadBalancer service on port 8000
- helm/heart-disease-api/ - equivalent parameterised Helm chart

The API is deployed to Docker Desktop Kubernetes. deployment.yaml runs 2 replicas with CPU/memory requests+limits, readiness and liveness probes on /health, and Prometheus scrape annotations; service.yaml exposes a LoadBalancer on port 8000 (reachable at http://localhost:8000 on Docker Desktop). A Helm chart provides the same deployment parameterised via values.yaml.

Commands:

```
kubectl apply -f k8s/  
kubectl get pods,svc          # wait until both pods are 1/1 Running  
curl.exe -X POST http://localhost:8000/predict -H "Content-Type:  
application/json" -d "@sample_request.json"  
# Helm alternative: helm install heart ./helm/heart-disease-api
```



Screenshot: Docker Desktop Kubernetes cluster enabled and Running (kubeadm, 1 node, v1.34.1).

```
PROBLEMS OUTPUT TERMINAL DEBUG CONSOLE PORTS SONARQUBE

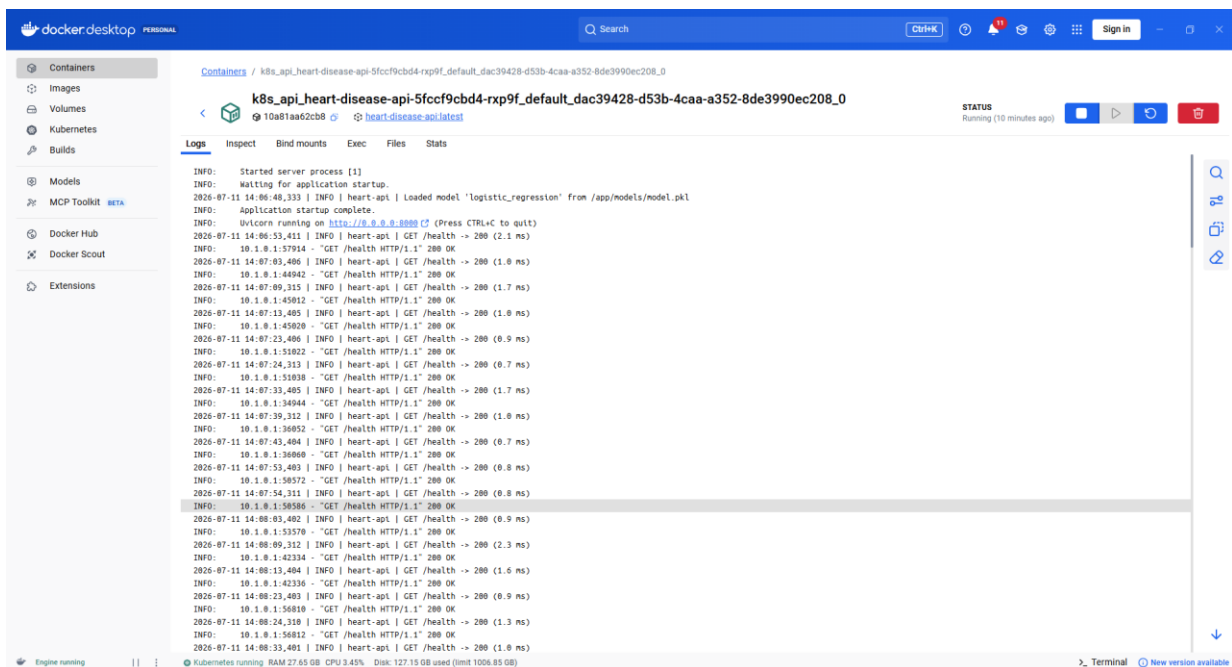
PS C:\workspace\Assignment\heart-disease-mlops> (Set-ExecutionPolicy -Scope Process -ExecutionPolicy RemoteSigned) ; (& c:\workspace\Assignment\heart-disease-mlops\venv\Scripts\Activate.ps1)
(.venv) PS C:\workspace\Assignment\heart-disease-mlops> kubectl config get-contexts
CURRENT  NAME          CLUSTER      AUTHINFO      NAMESPACE
*        docker-desktop  docker-desktop  docker-desktop
(.venv) PS C:\workspace\Assignment\heart-disease-mlops> kubectl config use-context docker-desktop
Switched to context "docker-desktop".
(.venv) PS C:\workspace\Assignment\heart-disease-mlops> kubectl cluster-info
Kubernetes control plane is running at https://kubernetes.docker.internal:6443
CoreDNS is running at https://kubernetes.docker.internal:6443/api/v1/namespaces/kube-system/services/kube-dns:dns/proxy

To further debug and diagnose cluster problems, use 'kubectl cluster-info dump'.
(.venv) PS C:\workspace\Assignment\heart-disease-mlops> kubectl get nodes
NAME                STATUS    ROLES    AGE   VERSION
docker-desktop      Ready    control-plane  101s  v1.34.1
(.venv) PS C:\workspace\Assignment\heart-disease-mlops> kubectl apply -f k8s/
deployment.apps/heart-disease-api created
service/heart-disease-api-service created
(.venv) PS C:\workspace\Assignment\heart-disease-mlops> kubectl get pods,svc
NAME                READY   STATUS    RESTARTS   AGE
pod/heart-disease-api-5fccf9cbd4-4bj2f  0/1     Running   0           9s
pod/heart-disease-api-5fccf9cbd4-rxp9f  0/1     Running   0           9s

NAME                                TYPE          CLUSTER-IP    EXTERNAL-IP  PORT(S)          AGE
service/heart-disease-api-service    LoadBalancer  10.111.221.53  <pending>    8000:32069/TCP   9s
service/kubernetes                  ClusterIP      10.96.0.1      <none>        443/TCP           3m6s
(.venv) PS C:\workspace\Assignment\heart-disease-mlops> kubectl get pods,svc
NAME                READY   STATUS    RESTARTS   AGE
pod/heart-disease-api-5fccf9cbd4-4bj2f  1/1     Running   0           88s
pod/heart-disease-api-5fccf9cbd4-rxp9f  1/1     Running   0           88s

NAME                                TYPE          CLUSTER-IP    EXTERNAL-IP  PORT(S)          AGE
service/heart-disease-api-service    LoadBalancer  10.111.221.53  <pending>    8000:32069/TCP   88s
service/kubernetes                  ClusterIP      10.96.0.1      <none>        443/TCP           4m25s
(.venv) PS C:\workspace\Assignment\heart-disease-mlops> █
```

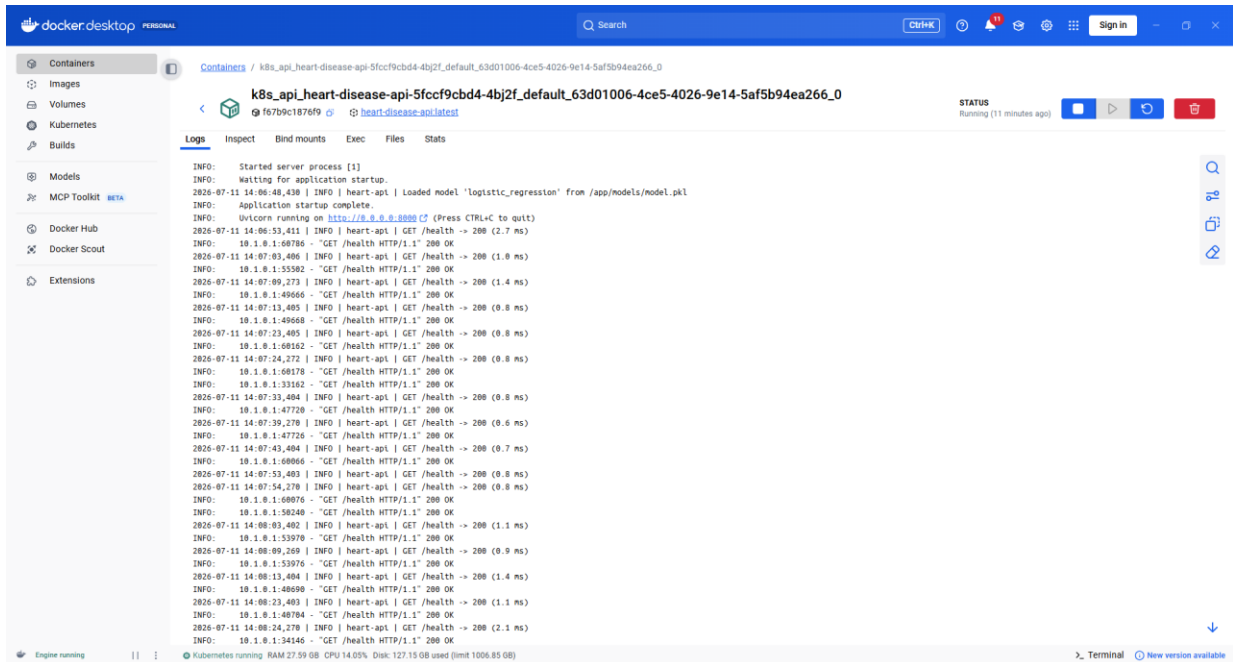
Screenshot: kubectl - context, cluster-info, get nodes (Ready), apply -f k8s/, and 2/2 pods Running behind the LoadBalancer service.



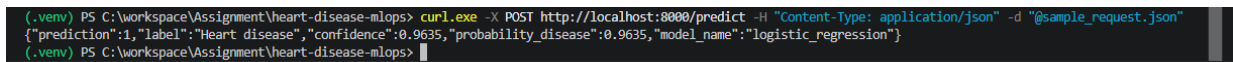
```
Containers / k8s_api_heart-disease-api-5fccf9cbd4-rxp9f_default_dac39428-d53b-4caa-a352-8de3990ec208_0
k8s_api_heart-disease-api-5fccf9cbd4-rxp9f_default_dac39428-d53b-4caa-a352-8de3990ec208_0
10a81aa62c8b heart-disease-api:latest
STATUS Running (10 minutes ago)

Logs Inspect Bind mounts Exec Files Stats
INFO: Started server process [1]
INFO: Waiting for application startup.
2026-07-11 14:06:48.333 | INFO | heart-api | Loaded model 'logistic_regression' from app/models/model.pkl
INFO: Application startup complete.
INFO: Uvicorn running on http://0.0.0.0:8000 (Press CTRL+C to quit)
2026-07-11 14:06:53.411 | INFO | heart-api | GET /health -> 200 (2.1 ms)
INFO: 10.1.0.1:53704 - "GET /health HTTP/1.1" 200 OK
2026-07-11 14:07:03.406 | INFO | heart-api | GET /health -> 200 (1.0 ms)
INFO: 10.1.0.1:44942 - "GET /health HTTP/1.1" 200 OK
2026-07-11 14:07:09.315 | INFO | heart-api | GET /health -> 200 (1.7 ms)
INFO: 10.1.0.1:45012 - "GET /health HTTP/1.1" 200 OK
2026-07-11 14:07:13.405 | INFO | heart-api | GET /health -> 200 (1.0 ms)
INFO: 10.1.0.1:45020 - "GET /health HTTP/1.1" 200 OK
2026-07-11 14:07:23.406 | INFO | heart-api | GET /health -> 200 (0.9 ms)
INFO: 10.1.0.1:51022 - "GET /health HTTP/1.1" 200 OK
2026-07-11 14:07:24.313 | INFO | heart-api | GET /health -> 200 (0.7 ms)
INFO: 10.1.0.1:51038 - "GET /health HTTP/1.1" 200 OK
2026-07-11 14:07:33.405 | INFO | heart-api | GET /health -> 200 (1.7 ms)
INFO: 10.1.0.1:34944 - "GET /health HTTP/1.1" 200 OK
2026-07-11 14:07:39.312 | INFO | heart-api | GET /health -> 200 (1.0 ms)
INFO: 10.1.0.1:36052 - "GET /health HTTP/1.1" 200 OK
2026-07-11 14:07:43.404 | INFO | heart-api | GET /health -> 200 (0.7 ms)
INFO: 10.1.0.1:36060 - "GET /health HTTP/1.1" 200 OK
2026-07-11 14:07:53.403 | INFO | heart-api | GET /health -> 200 (0.8 ms)
INFO: 10.1.0.1:50572 - "GET /health HTTP/1.1" 200 OK
2026-07-11 14:07:54.311 | INFO | heart-api | GET /health -> 200 (0.8 ms)
INFO: 10.1.0.1:50586 - "GET /health HTTP/1.1" 200 OK
2026-07-11 14:08:03.402 | INFO | heart-api | GET /health -> 200 (0.9 ms)
INFO: 10.1.0.1:53570 - "GET /health HTTP/1.1" 200 OK
2026-07-11 14:08:09.312 | INFO | heart-api | GET /health -> 200 (2.3 ms)
INFO: 10.1.0.1:42334 - "GET /health HTTP/1.1" 200 OK
2026-07-11 14:08:13.404 | INFO | heart-api | GET /health -> 200 (1.6 ms)
INFO: 10.1.0.1:42336 - "GET /health HTTP/1.1" 200 OK
2026-07-11 14:08:23.403 | INFO | heart-api | GET /health -> 200 (0.9 ms)
INFO: 10.1.0.1:56010 - "GET /health HTTP/1.1" 200 OK
2026-07-11 14:08:24.310 | INFO | heart-api | GET /health -> 200 (1.3 ms)
INFO: 10.1.0.1:56012 - "GET /health HTTP/1.1" 200 OK
2026-07-11 14:08:33.401 | INFO | heart-api | GET /health -> 200 (1.0 ms)
```

Screenshot: Kubernetes pod logs (replica 1) - model loaded, readiness probes returning 200.



Screenshot: Kubernetes pod logs (replica 2) - model loaded, health checks passing.



Screenshot: /predict verified through the Kubernetes LoadBalancer service.

10. Task 8 - Monitoring & Logging

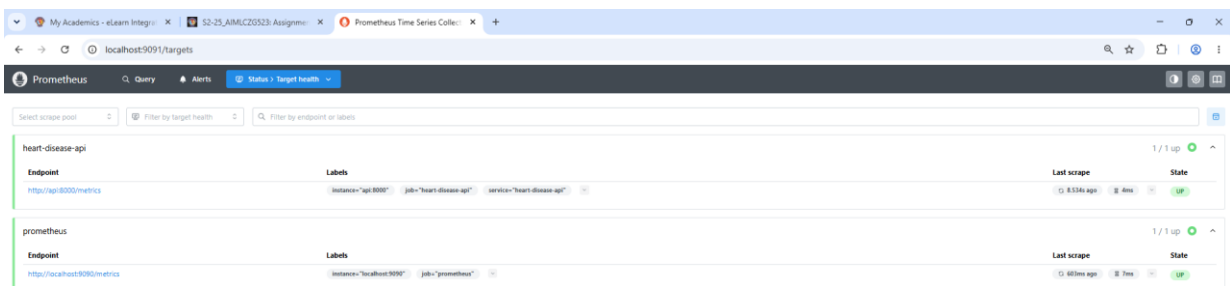
Source files:

- `src/api/main.py` - request-logging middleware + Prometheus metrics
- `(Instrumentator /metrics + heart_predictions_total{outcome=...} Counter)`
- `monitoring/prometheus.yml` - scrape config (api:8000 every 10s)
- `docker-compose.yml` - API + Prometheus + Grafana stack
- `monitoring/grafana/provisioning/` - auto-provisioned datasource + dashboard

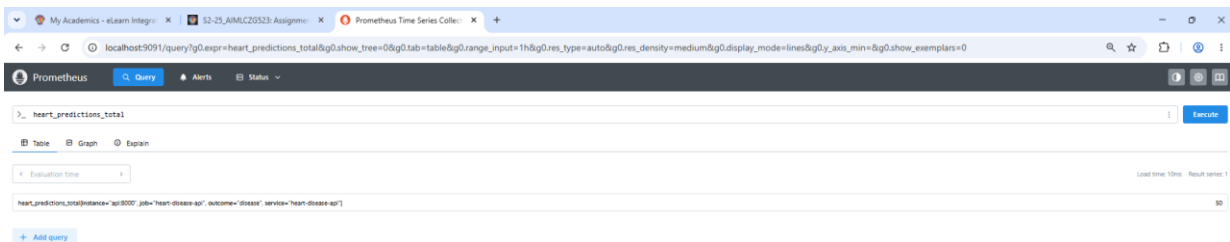
A FastAPI middleware logs every request (method, path, status, latency). prometheus-fastapi-instrumentator exposes standard HTTP latency/throughput metrics at /metrics, plus a custom `heart_predictions_total{outcome=...}` counter that tracks the positive/negative prediction mix (a simple drift signal). docker compose brings up the API (host port 8001, to coexist with the K8s deployment on 8000), Prometheus (host 9091), and Grafana (host 3000) with the datasource and dashboard auto-provisioned.

Commands:

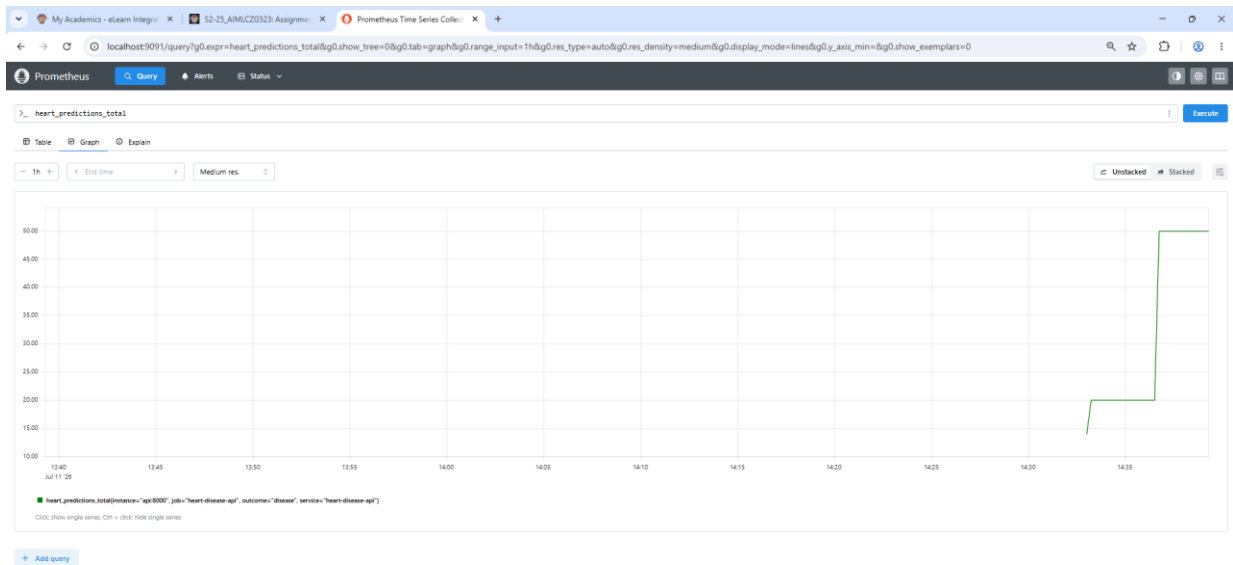
```
docker compose up --build
# generate traffic (API on 8001):
1..30 | ForEach-Object { curl.exe -s -X POST http://localhost:8001/predict -H
"Content-Type: application/json" -d "@sample_request.json" | Out-Null }
# Prometheus: http://localhost:9091   Grafana: http://localhost:3000
(admin/admin)
```



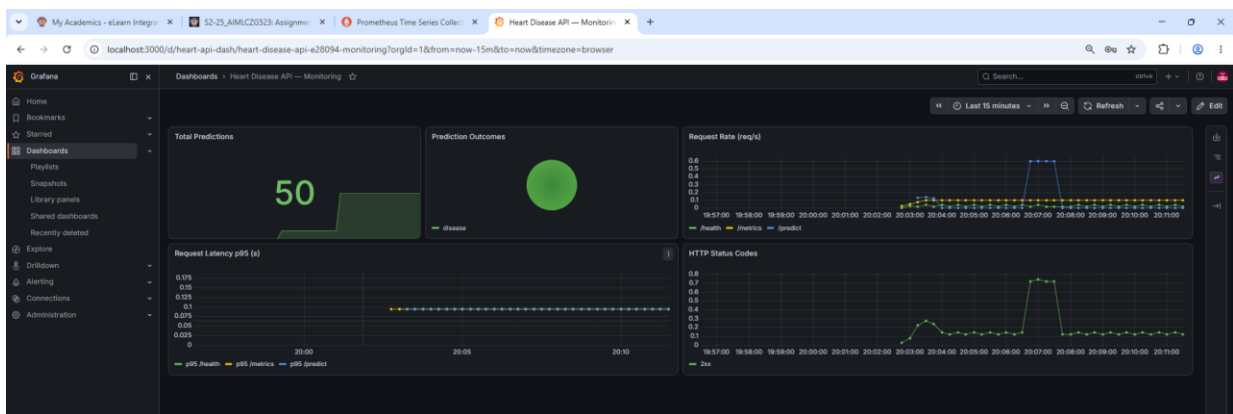
Screenshot: Prometheus - both scrape targets UP (heart-disease-api scraped at api:8000/metrics).



Screenshot: Prometheus query - heart_predictions_total = 50 for outcome='disease'.



Screenshot: Prometheus graph - `heart_predictions_total` over time.



Screenshot: Grafana dashboard - Total Predictions, Prediction Outcomes, Request Rate, p95 Latency, HTTP Status Codes.

11. Task 9 - System Architecture

Development produces a tracked, versioned model artifact; CI/CD validates and packages it into a Docker image; Kubernetes serves it behind a LoadBalancer; Prometheus and Grafana observe it in production.

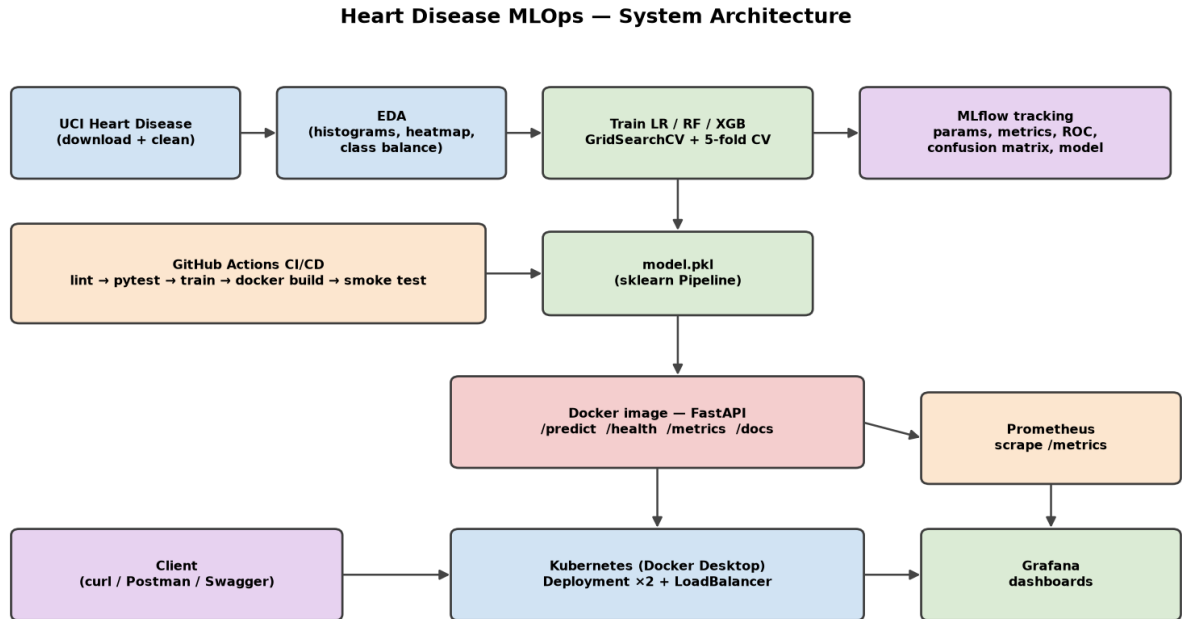
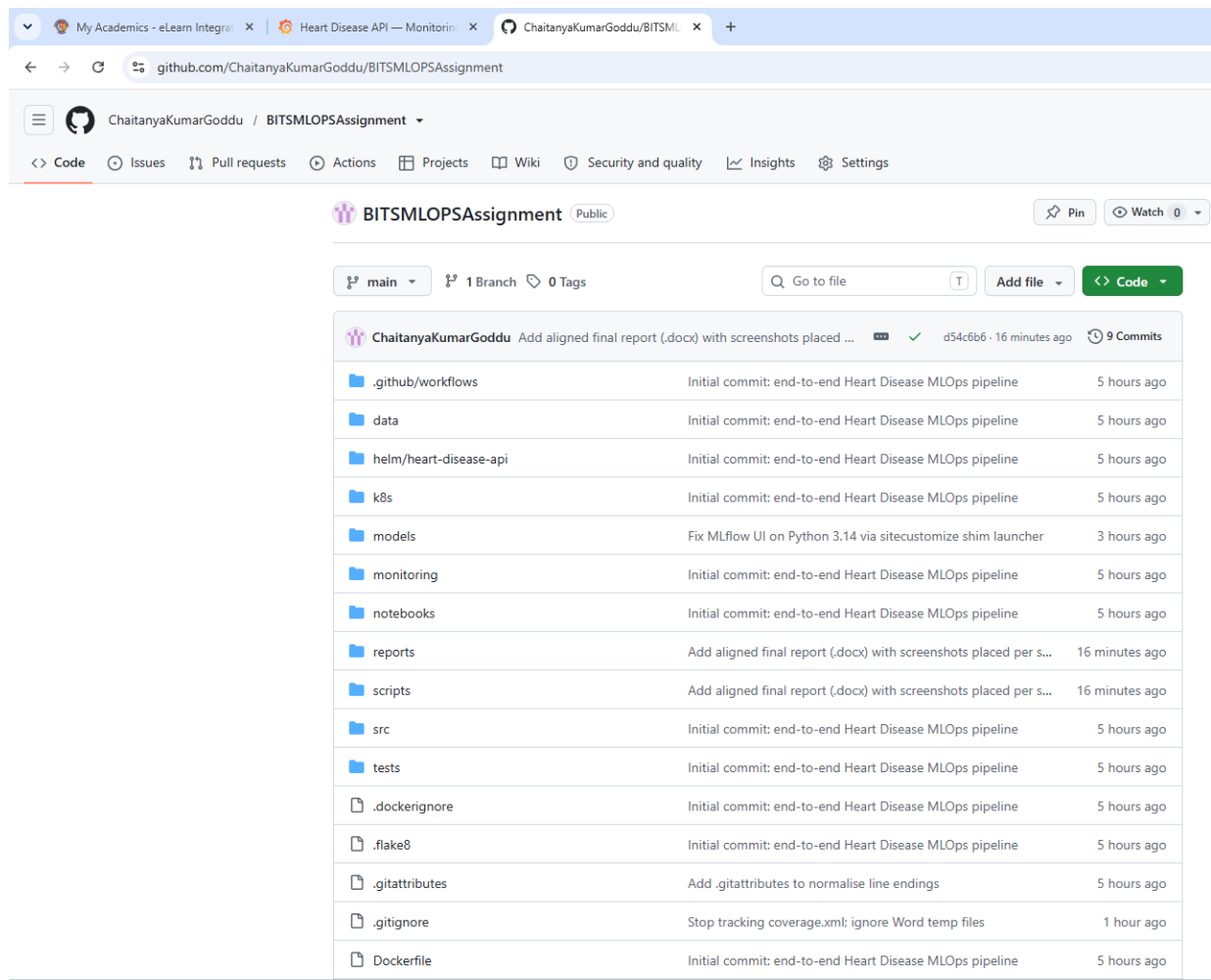


Figure 7. End-to-end system architecture.

12. Repository & Deliverables

GitHub repository: <https://github.com/ChaitanyaKumarGoddu/BITSMLOPSAssignment>



Screenshot: GitHub repository - project structure on the main branch.

Deliverable	Location in repo
Code + Dockerfile + requirements	repo root, src/
Dataset + download script	src/data/download.py, data/processed/
Notebook / scripts (EDA, train, inference)	notebooks/, src/
Unit tests	tests/
GitHub Actions workflow	.github/workflows/ci.yml
Deployment manifests / Helm chart	k8s/, helm/
Monitoring config	monitoring/, docker-compose.yml
Report + figures	reports/

13. Conclusion

This project delivers a reproducible, tested, containerised, and monitored heart-disease classifier that meets every assignment requirement. The tuned Logistic Regression (ROC-AUC 0.968, recall 0.93) is both accurate and interpretable - the right choice for a clinical screening context - and the surrounding MLOps scaffolding (MLflow, CI/CD, Docker, Kubernetes, Prometheus/Grafana) makes it safe to iterate on and operate in production. Possible extensions: data-drift detection with Evidently, a model registry with staged promotion, automated retraining triggers, and canary deployments.